RHAPSODY

RHAPSODY BEST PRACTICE GUIDE

Version 6.10.1

Professional Services Team, APAC

3 September 2024, Version 1.0

DOCUMENT HISTORY

Version	Date	Author(s)	Comment
0.1	8/03/2022	Francis Mullane	Initial rebranding based on 6.5 guide
0.2	6/06/2023	Francis Mullane	Updated for 6.10
0.3	3/09/2024	Francis Mullane	Updated to 2nd rebrand back to Rhapsody with new styles, removed references to Lyniate

REFERENCES

Title, Version	Author / Producer	Enclosed As
Rhapsody Documentation Portal	Rhapsody Product Team	Reference
Rhapsody Product Jira	Rhapsody Product Team	Reference

CONTACT DETAILS

For more information regarding this document, please contact:

Francis Mullane – Senior Solution Architect
Rhapsody

Email: francis.mullane@Rhapsody.Health
Website: www.Rhapsody.Health

InterOperability Health (Australia) Limited
60 Margaret Street, Floor 14
Sydney, NSW, 2000

Table of Contents

1 Purpose 4

1.1 Benefits 5
1.2 Use of this Guide 6
1.3 Terms and Abbreviations 6

2 Reading Order 8

3 Rhapsody Integration Engine Best Practice Principles and Guidance 9

2.1 Handling of Exceptions to or Deviations from Best Practice 10
2.1.1 Document the Reason/Requirement 10
2.2 Check in Documentation (Comments) 11
2.3 In Route Documentation 11
2.4 Naming Conventions 14
2.4.1 Component Names 15
2.4.2 Folders and Lockers 15
2.4.3 Baseline guide for Naming Conventions 23
2.5 Security and Access Controls 31
2.6 Route Design 32
2.6.1 Discard Before Processing 32
2.6.2 Fail-safe Fail-early 32
2.6.3 Intra-Route Error Handling and Exception Processing 33
2.6.4 Route Layout and Processing Complexity 35
2.6.5 Linking/Chaining 38
2.6.6 Message Definitions and Parsing 40
2.6.7 Multi vs. Single Threading 43
2.6.8 Special consideration for use of dynamic routers 45
2.7 Queue Depth 48
2.8 Communication Point Considerations 50
2.8.1 Advanced Routing 52
2.8.2 HTTP Communication Points 52
2.9 Filter Considerations 54
2.9.1 General Considerations 54
2.9.2 Use of Conditional Connectors between Filters 55
2.10 JavaScript Use and Features 58

2.10.1 Code Guidelines 58
2.10.2 Use of Shared JS Library function 59
2.10.3 Considerations for Multiple JS Filters regarding Parsing/Route Performance and storage 61
2.11 Message Mapping Options and Considerations 62
2.11.1 General Considerations 62
2.11.2 Mapper Filter Specific considerations 64
2.12 Database Considerations 65
2.12.1 Where Possible Use the Database Communication Point in Preference to Filters 65
2.12.2 Choose the Correct Database Components 65
2.12.3 Use Configuration Properties to Handle Connection Failures Properly 66
2.12.4 Use Notifications 66
2.12.5 Use Stored Procedures and Views in Preference to SQL Statements 67
2.12.6 General SQL Scripting Standards 67
2.12.7 Rhapsody Lookup Tables 69
3.13 Developing Custom Components with the RDK 71
2.14 Message Properties 73
2.14.1 Avoid Assignment of Message Body to Property 74
2.14.2 Managing Properties 75
2.14.3 Indexing and Searching 76
2.15 Rhapsody Variables 77
2.16 Testing and Peer Review 78
2.16.1 Development Testing 78
2.16.2 End to End / Route Testing 79
2.17 Other/General Considerations 80
2.17.1 Assign a Specific User to Run the Rhapsody Process 80
2.17.2 Use Individual Accounts for Access to the IDE and Management Console 81
2.17.3 Backups 81
2.17.4 Archive Cleanups 82
2.17.5 Configure Management Console Notification Schemes 82

4 Virtualisation Best Practice 83

1 Purpose

The purpose of this document is to outline best practice guidelines for use in design, implementation, peer review, and customer sign off of Rhapsody based solutions that are delivered to Rhapsody Health customers by Professional Services APAC, our partners, or by the customer themselves.

This document aims to ensure consistency in delivery of customer solutions that have similar sets of requirements, or integration patterns, regardless of the team delivering the solution. Additionally, it aims to reduce delivery effort in evaluating multiple potential approaches where the optimal approach is already proven by past delivery efforts.

The guidance provided in this document is not intended to be prescriptive or taken as an absolute mandate as there may be situations where a given solution is required to deviate from the recommended approach to fulfil the solution requirements.

By adhering to best practice guidelines wherever possible we seek to ensure that our solutions are:

- Error-free (configuration should implement the specification correctly)
- Delivering messages reach the correct destination in the correct form
- Handling messages which invoke an exception during processing, so that they are identifiable and processed appropriately
- Processing of messages through the engine occurs in a timely manner (as required by the system specification)
- Documented consistently
- Aligned with the intended use of Rhapsody

1.1 Benefits

Table 1: **Benefits Description**

Benefit	Description
Consistent Delivery Patterns	Consistency across solutions and customers so that PSG, Customer and Support teams can readily understand the as-built solutions.

Reduced Effort	Rhapsody offers multiple tools that may be used to solve the same types of problems. By formalising the known optimal patterns, time and effort testing between possible solutions is reduced.
Optimal Outcomes	Leveraging the experience of the Professional Service Team as documented in this guide ensures optimal solutions and avoids known bad patterns that have resulted in support issues or sub-optimal processing and performance noted in Health Check reviews.

1.2 Use of this Guide

This is a living document – principles and best practice change and evolves over time due to:

- New or improved functionality within Rhapsody or its underlying JVM architecture
- Improved engine and component efficiency
- Newly adopted and field-tested patterns which better address solutions to customer requirements in novel and beneficial ways; or which have been adopted as a means of resolving identified issues from past practices

This version of the Best Practice Guide is aligned to version 6.9 of the Rhapsody Integration Engine, and it should be noted that some of the recommendations contained within this scope may not be relevant, or in some cases possible, on earlier versions of Rhapsody. It is expected that this document will continue to be updated based on product releases and practical input from Professional Services, Project Support, and Product resources worldwide.

Also, due to the substantial improvements to functional capability, throughput and efficiency that have been seen in past major version upgrades, as part of our best practice guidance Rhapsody Health strongly recommend our customers to deploy either the n-1, or latest version, based on review of documented bug fixes, known issues and new features.

1.3 Terms and Abbreviations

Term/Abbreviation	Definition
DBA	Database Administrator
DR	Dynamic Router Rhapsody communication point
E4X	ECMAScript for XML. This is a programming language extension that adds native XML support to JavaScript
FIFO	First In First Out
HAPI	HL7 application programming interface. This is an object-oriented HL7 2.x parser for Java. Within Rhapsody 6.9 HAPI version 2.2 is used which provides support for HL7 2.x up to v2.6
HL7	Health Level 7
JS	JavaScript
JSON	JavaScript Object Notation
JVM	Java Virtual Machine
LTS	Long Term Supported release
Management Console	Web-based application used to sort, view, edit and re-send messages processed the Rhapsody Integration Engine (Rhapsody)
PS	(Rhapsody Health) Professional Services
PSG	(Rhapsody Health) Professional Services Group
RDK	Rhapsody Development Kit used to create custom Rhapsody components
Rhapsody	Rhapsody Health's integration engine
Rhapsody IDE	A Windows application for configuring the Rhapsody engine to build interfaces between different systems
SOA	Service-Oriented Architecture
XML	eXtensible Markup Language

2 Reading Order

Beginning with the new Version 7 of the RIE the guide is now produced as a set of addendum documents when required. The v 6.10 Rhapsody Best Practice Guide should be read first as the general principles and consideration still apply to version 7.x. Additionally it is expected that the PSG team will need to engage regularly with customers who are still on v6.x for new interface delivery, upgrades and HealthCheck Reviews.

The version 7.x addendum(s) will focus on containing guidance where version changes have introduced new features, improvements or changes that require similar, targeted, guidance to ensure consistent use of a new feature or where improvements and product changes have negated a pattern used in version 6 (e.g. changes to XML handling, JavaScript standard functions etc.

Important Known Issues and bugs that have been found with early adopter customer engagements by PSG APC will also be noted in this document to assist with customer engagements (avoiding issues, providing upgrade advice for versions where issues are known and cannot be avoided for a required interface workflow).

3 Rhapsody Integration Engine Best Practice Principles and Guidance

3.1 Handling of Exceptions to or Deviations from Best Practice

As highlighted in the Introduction above, this document is provided as a guideline only. It will not always be possible to follow the recommendations because for a given solution:

- There may be scenarios where adhering to one best practice principle is at odds with another - in such situations, we would adhere to the principle of greatest importance for that scenario
- A novel solution may be required that requires a pattern that has not been widely encountered and has not been considered at the time of writing of the guide
- There may be other practical or licensing constraints, or directives from a customer's project team that require us to deviate from best practice

These types of scenarios should rarely be encountered, should be considered carefully, and should always be managed in a consistent fashion. Therefore, where a deviation is required, the following process will be adhered to (without exception) by the team implementing the solution.

3.1.1 Document the Reason/Requirement

Once a new potential requirement to deviate is identified, and the pattern is not one already known to require a deviation (i.e., has not previously been encountered, reviewed, and accepted), the requirement and the proposed approach will be documented and distributed via Rhapsody Health/Customer peer review and discussion channels to:

- Ensure that the approach is sound

- Allow opportunity for wider discussion and review from a larger pool of experienced stakeholders to ensure all possible alternatives that would align with best practice have been considered
- Ensure future similar patterns that require a deviation from best practice will be handled in a consistent manner and will still be deployed in a similar manner for a similar requirement
- Ensure the proposed interface design is in alignment with the intended use of Rhapsody

Where the requirement meets a known pattern for deviation, this will be documented, and a similar proven approach to resolving the issue as has been taken previously will be adopted.

If the solution could have been deployed via a best practice approach but this is not possible due to licensing or customer constraints/requirements, the decision made and the reasoning behind the decision to not pursue the best practice approach will be documented in (at minimum):

- The Decision Log of the project manager
- The Route Note documentation feature within Rhapsody IDE

3.2 Check in Documentation (Comments)

When multiple team members share an environment, ensure that adequate comments/documentation are provided when checking in changes.

It is beneficial to follow a standard format agreed for the project. For example:

Description of what has changed
Issue: CPO-123456
Reviewer: Mickey Mouse

3.3 In Route Documentation

All Routes and Components, where appropriate, will contain the following minimum documentation within their Notes. For shared JS Library functions, JS Filters and Mapper Filters this will be included at the start of the code and a summary will be included within the Route Notes section.

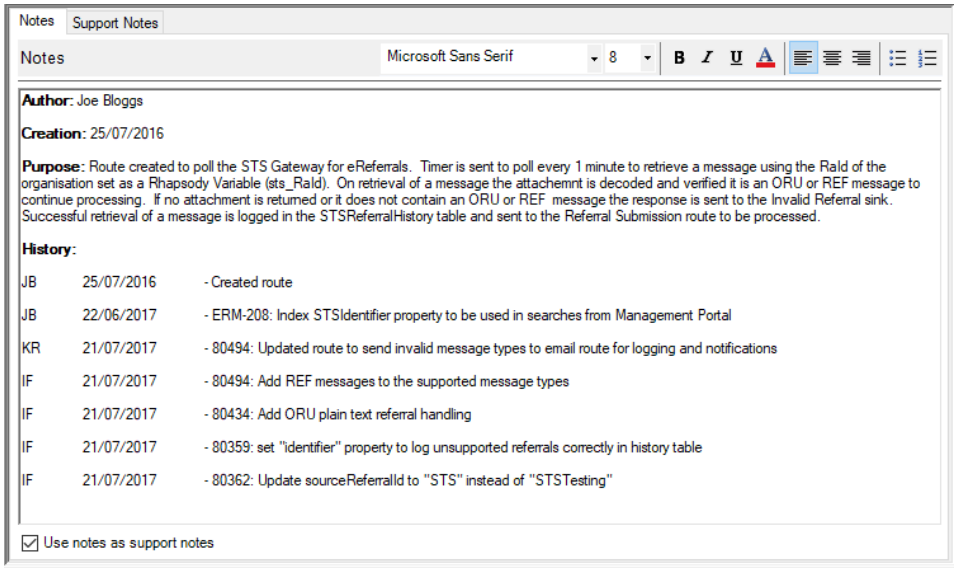
Item	Value

Author	The person initially configuring the component. Note APAC PSG typically use the following convention:
	Author : Rhapsody - <firstname> <lastname> e.g.
	`/* Author : Rhapsody - Francis Mullane`
	`Create Date : August 2024`

Creation	The date of initial creation
Purpose	The description of the purpose of the component. This should contain all necessary information to understand what the component is deployed to achieve and a summary of how that is achieved
History	Modification history of the component after development (e.g. BAU, fix issue found in UAT). One line per change, each line contains the initials or name of the person making the change, the date, and the change description (include feature/bug ticket reference if available)

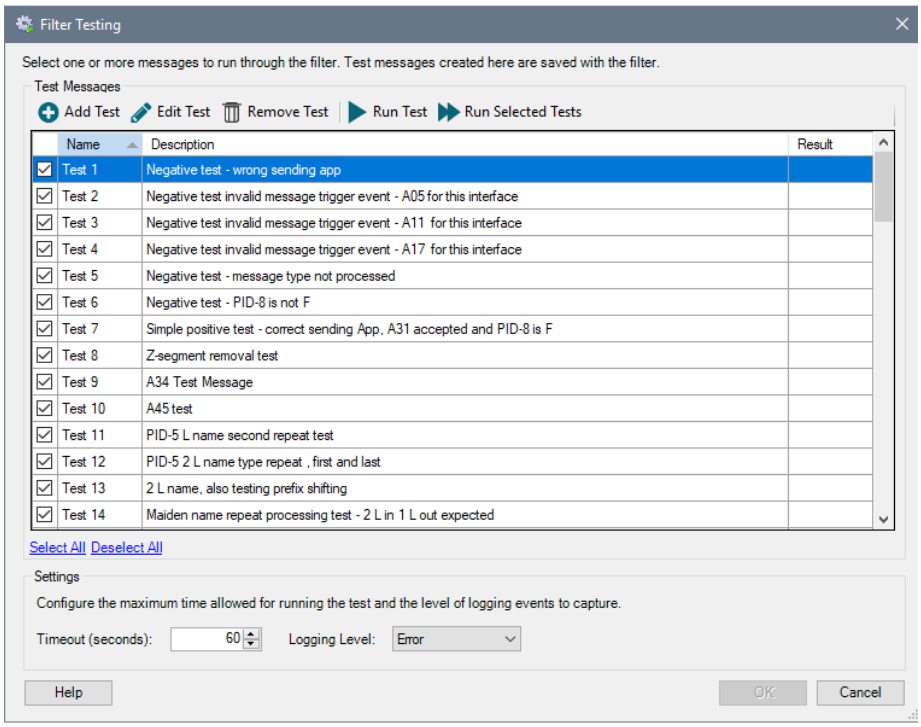
Adding comments as explained above will not only aid ongoing maintenance but can also be viewed in Rhapsody's self-generated documentation. See Figure 1 below for an example of route documentation.

Figure 1: **Route Documentation in Rhapsody IDE**



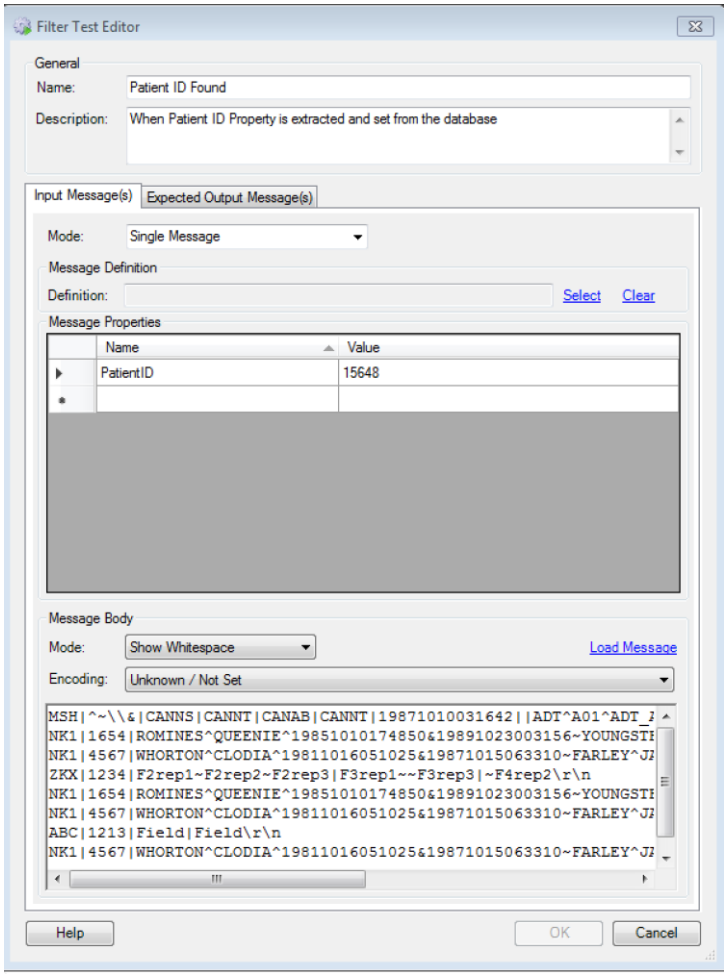
Additionally, any test messages that are used during unit testing must be saved with a meaningful name and concise description of the test case, as shown in Figures 2 and 3 below. This enables other users to peer review and retest using the same test messages.

Figure 2: **Filter Test Cases**



Note to group sets of alternate, and related test cases, together the Name Column may be specified as Test 1.a , Test 1.b etc.

Figure 3: **Test Message within a Test Case**



Note that there should be sufficient test cases to satisfy unit testing of each permutation and combination of positive and negative test results, frequently errors in logic may be hidden at build time and carried over to other environments, potentially Production, if there are not sufficient cases to trigger all logic branch executions. In general test cases will be added at the same time as each piece of configuration / logic condition is added.

TIP : When first configuring test cases if you configure the input message as the "Expected Output" message then during testing the changes and transformations are highlighted as differences making them easier to inspect in the UI.

3.4 Naming Conventions

Naming conventions are focused at allowing the implementation of configurations which are essentially self-documenting; they should enable

components to be labelled in a manner which is clear and indicative of purpose. This allows grouping of components and simplifies searches across components in various UI components which streamlines the development and management processes. The naming of Communication Points and Filters with information such as their type and purpose is especially important when the components are viewed in the Management Console where they are not distinguishable by different icons (unlike the IDE which has different icons for different component types).

Consistency in naming simplifies maintenance by both internal and external resources as it reduces the time needed to understand the purpose of the configuration.

Naming conventions will be discussed and agreed with the customer prior to commencement of implementation and are expected to be adhered to by all team members working on a given solution. The salient point for this best practice is to ensure you have an agreed and consistent naming convention prior to the commencement of a solution build.

The following represents Rhapsody Health's current recommendation for naming conventions, as well as the rationale for these recommendations (modification to this recommendation may be required on a case-by-case basis and will be handled by the process outlined in Section 2.1).

3.4.1 Component Names

Components should be named per its function and purpose. Activities which benefit from a sound naming convention include:

- Searching for a component in the Management Console.
- Applying a hierarchy to components displayed within the Rhapsody IDE.
- Analysing and understanding the processing logic.
- Streamline on-boarding and knowledge transfer

3.4.2 Folders and Lockers

3.4.2.1 Folders

Folders group a set of related components. They are utilised in the IDE (and Management Console) to provide structure and organisation to the configuration. Use of a structure and meaningful names is important for the following reasons:

- Mitigate long route and communication point names and provide an easily searchable structure
- High-level indication of function/purpose of routes held under the folder (for example, a folder might be used to group processing logic for input from a specific host; the host system type might be included in the folder name)
- Indicator of source and/or target system for messaging for the child routes (for example PAS_ADT or PAS_to_RIS)

The folder structure must be aligned to the requirements of a solution. There are two general folder structures that are most used: SOA/De-Coupled Paradigm and Interface/Solution Paradigm. However, a certain level of flexibility in the folder structure may be required to ensure alignment with the solution requirements.

The SOA / Decoupled Paradigm is the pattern that is followed by Rhapsody Health implementations, as most large-scale deployments fit this pattern. It is important for maintainability that as the number of interfaces grows, a consistent approach is taken so that resources can easily identify components that require fixers, modifications or extensions for new requirements. Thus, the decision has been made to adhere to the SOA / Decoupled pattern whenever possible, unless required to do so to meet existing customer requirements.

Interfaces which do not fit this pattern, or require a different approach for considerations such as handling large files with fewer steps to reduce data store impacts, will be deployed within the 'Non-Standard' folder of the deployment.

3.4.2.1.1 SOA / Decoupled Paradigm

Folders are organised under the respective locker by system name (e.g. PAS, LIS etc.), each folder will have subfolders that allow grouping of routes and related configurable components such as (folders which are not required can be omitted):

- Input
- Processing (such as Normalisation, EMPI enrichment, identifier resolution, common code translation etc.)
- Output

- Common (for components that are shared by some or all the above such as a Dynamic Router, Web Service client, etc.)
- Non-Standard (for interfaces / components that are part of the solution, but which do not follow the same input>Processing>Output pattern as the majority of other interfaces

Figure 4: **Figure 1 Example SOA/Decoupled Folder Structure**

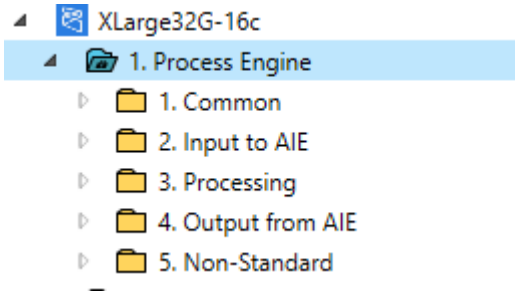
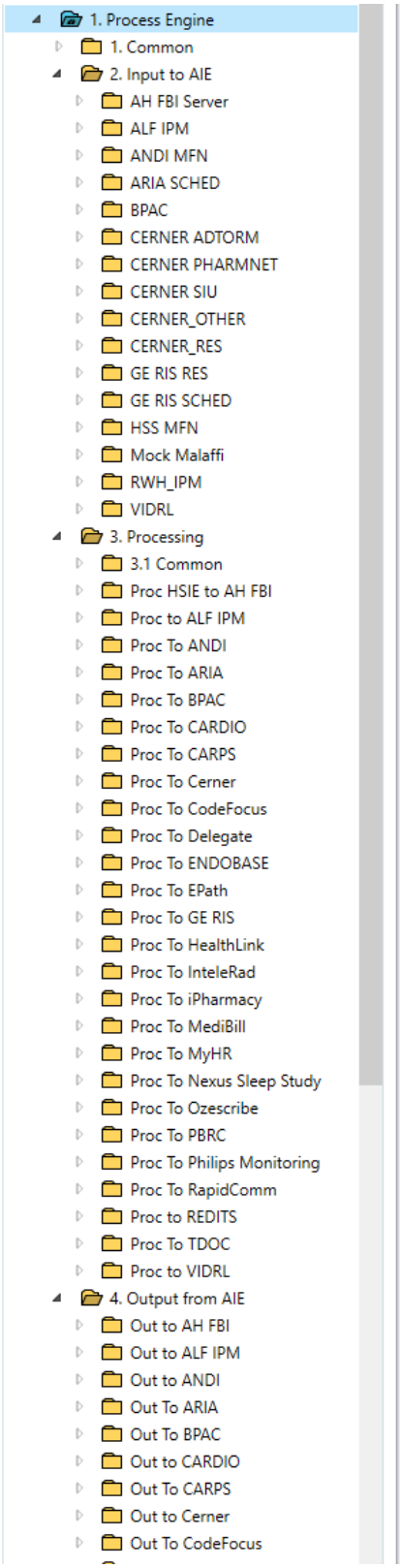


Figure 5: **Example SOA/Decoupled Folder Structure expanded**



Advantages

- Current solution components are easily identifiable based on the source, destination or any processing requirements making it easy to identify common components (such as the input part of an interface) that can be

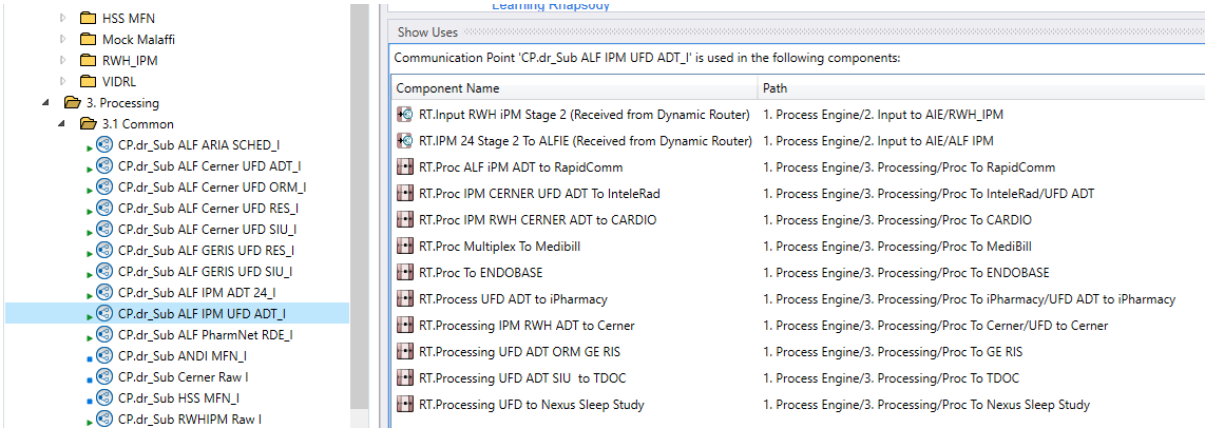
reused within new interfaces that may share one or more of the existing Producing, Consuming or Processing components. For example in an environment that has a Common Messaging Model for ADT a new Producer may be added which publishes normalised ADT messages that can be processed and send to downstream consumers using existing configuration in the Processing and Output folders

- This approach supports decoupling of systems participating in an end-to-end interface as Rhapsody is the mediator and source or destination may be replaced (removed or added) with restricted changes to a set of components required for the new system.

Disadvantages

- It is not always apparent to developers/administrators within the IDE as to which components contribute to an end-to-end interface as the components laid out in different folders.
  - This disadvantage may be mitigated through site specific documentation, including best-practice route note documentation.
  - Additionally this disadvantage may be partially mitigated through the use of a combination of static destination targets in the Publishing routes and Subscribing input Dynamic Routers with the setting for Unique Target Name selected. This then enables identification of all interfaces that process a given type of data through the "show uses" feature e.g. in the following diagram all interfaces that process the normalised PAS (iPM) ADT are shown:
    - NB static destination + Unique Target pattern is only recommended in scenarios where downstream subscribers retain the majority of messages. Where the downstream receivers discard the majority of messages this can lead to over-utilisation of the datastore. For the latter scenario use a combination of Dynamic Destinations for the Publishing routes and Unique Target Name for the subscribers (combined with careful naming of the Targets to aid in identification of the data type / Pub/Sub relationship)

Figure 6: **Identifying Related Interfaces via Show Uses**



- Careful use of subfolders and/or route naming within the Processing and Output components may also be used to help identify the relationship between the components and the interfaces they instantiate as shown in the following example figures:

Figure 7: **Example use of Folder / Route naming to identify the different data types where processing requirements are not common**

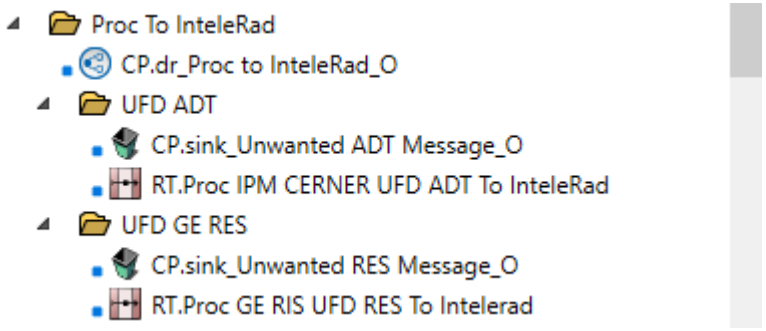


Figure 8: **Example where a common processing component is used for multiple data types / interfaces**

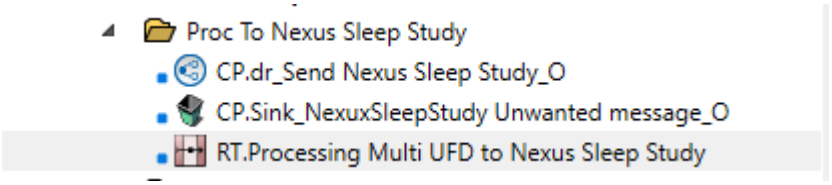
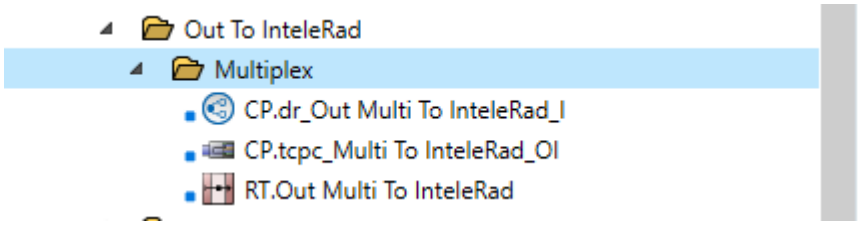


Figure 9: **Example where a common output component is used for multiple data types/interface**



3.4.2.2 Lockers

A locker is used for compartmentalising the configuration in a way that allows access to that configuration to be controlled.

Our best practice guidance with respect to this is to determine the locker requirement on a project-by-project basis – lockers can be added as required:

- To segregate components to ensure team members are not able to modify components that are not related to their solution or tenancy,
- To ensure that administrative users are only able to access a subset of message data and components within the management console (e.g. to restrict access to patient data only from their site, or to ensure they are only able to start/stop components under their jurisdiction).

Messages may be directed from one Locker to another however they are only able to be distributed between lockers via the use of Dynamic Router communication points.

Message snapshots cannot be reloaded in lockers other than the locker that they were captured. This is to ensure that users in other lockers are not able to load a message they are otherwise not entitled to access, thus the use of Lockers may at times require other suboptimal configuration such as setting a message body at a point in time as a property, so it is available in processing within other lockers (this can greatly impact the data store use for certain deployments compared to the use of Message Snapshots).

We do not recommend segregation into lockers unless there is a driving requirement to do so as it increases the maintenance effort, and due to the impact to use of message snapshots noted above. In particular Lockers should not be used to simply segregate configuration and should not be used to group common components used in multiple other lockers/ folders.

Usage of multiple lockers requires the following components and related configuration to be within the same locker for messages to flow correctly. As such, they will need to be duplicated into different lockers and this results in additional efforts to maintain the components in multiple locations (e.g., to add/change a message type in a definition):

- Communication Point
- Message Definition

- Message Tracking Scheme
- Chained/Linked Route

3.4.3 Baseline guide for Naming Conventions

The following is our typical starting point for discussion of naming conventions, in general this will cater to most deployments but may require modification to suit the needs of an implementation – in either case the implementation team is expected to adhere to these agreed conventions to ensure consistency in development, support and ongoing management between the teams involved.

3.4.3.1 Separator Characters in Names

Components may require medium length strings of multiple tokens (including prefixes and suffixes) to indicate use and purpose as described below. Generally, it is recommended to separate these by either the space character or the underscore character. The following general considerations should be noted when naming components:

- The underscore is only suitable for shorter token sequences – for longer sequences, particularly when naming items that appear within the Route Canvas (Communication Points and Filters), the use of the space character is recommended to aid in succinct canvas layout.
- Use of the underscore in component names tends to stretch the component name horizontally and can make efficient use of layout in the canvas difficult
- Use of underscores in filenames such as Message Definition and Mapper files has no impact on layout within the IDE
- Try to avoid component names which would wrap more than 3-4 lines in the Route Canvas
- Try to avoid route names which are long, or nesting folders too deeply as this makes it difficult to see the entire component name without the need to scroll within the IDE Workspace Pane.

Table 2: **Naming Conventions**

Component	Format	Description	Examples
Lockers			
	<Cluster> or <Tenant> or <Function>	Lockers are named based on the requirement for the locker (e.g. to segregate by the role of the interfaces within the Locker, or the separation requirement such as a multi-agency tenancy which would have a Locker per tenant)	DHHSIE QHIP Core Complex Conductor Process Engine Services
Folders			
	##_<Folder type>	Folders should be named according to pattern that is adopted (SOA vs Interface / Solution patterns) **SOA Folder Structure** Top level folders are named according to their purpose / use in the solution i.e. Common, Input, processing, Output, Non-Standard.	**SOA:** 01.Common\Logging 02.Input\Cerner\ADT \ORM Or 01.Common\Logging 02.Input\CernerADT \CernerORM

Second level folders are typically named based on the System involved e.g. a Cerner sub-folder of Input indicates there will be components within the folder that receive messages from Cerner.

Third level folders may be used to further separate components where there are multiple interfaces involved from/to the same systems (alternatively you can use multiple appropriately named Second level folders). For example, within a second level Processing to Cerner folder there may be folders for 'Processing iPM to Cerner' and 'Processing GERIS to Cerner'.

Folders may be prefixed to apply an ordering within the IDE / Management console e.g. to ensure Input appears before Output.

Interface / Solution Folder Structure
Folders should be named according to either:

		• The primary source or destination systems they integrate with Or • The primary function the routes perform The folders may be prefixed with a sequence number to indicate the different stages of processing	
Routes			
Prefix (Optional) Suffix (Optional)	RT.##_<name> RT.<name> Stage ##	Configurations may break a complex processing sequence across several route stages; in these cases, it may be of assistance to include the sequence ID in the route name to assist understanding the processing flow. • All direction specification should be Rhapsody-centric; that is, Input means Rhapsody is receiving the message while Output means Rhapsody is sending the message. • For the body make it clear what the purpose of the component is	RT.01 iPM ADT to Platform RT.02 map iPM Codesets RT.03 transform iPM to CMM RT. iPM ADT to Platform Stage 1 RT.iPM ADT Codesets Stage 2

Communication Points			
Type Prefix Direction Suffix	CP.<type>_description _<direction>	Prefix identifies the type of filter Description identifies the purpose Suffix indicates how the message traverses the Communication Point _I – Input _O – Output _OI – Out -> In _IO – In -> Out _BI - Bidirectional	CP.db_Endobase Notes_OI CP.dir_Holding Files_O CP.tcps_Receive iPM ADT_IO CP.tcpc_Send Orders to GERIS_OI CP.dir_Move Cerner Files_BI
Filters			
Prefix	<type>.name /description	Type is a solution consistent abbreviation of the Filter type (used to identify different filter types in management console paths)	js. Filter and transform iPM ADT to GERIS js. Collect and re-order messages map. Map iPM ADT to GERIS b64.encode ORU db. Lookup PAS Codes dd. Detect Duplicate PAS Message nop. Connects to Rt.PAS Stage2 pp. Set Inbound Properties

			ss. Snapshot original Inbound Message ex. Execute PDF Process bdb. Batch generated files x2p. Generate output PDF
No-operation filters used as bookends to connect routes	FROM_<preceding route> TO_<next route>	Generally the use of No Operation filters for route descriptions or bookends is discouraged, however in the use case where routes are directly linked updating / deleting / re-organising operating filters can lead to the connection/ relationship being lost. For this reason NOP filters placed as the last filter of a FROM route and the first filter of a TO route should be used to join the routes when using route-to-route connections	nop.TO_RT.ADT Processing Stage 2 nop.FROM_RT.ADT Processing Stage 1
Conditional / JavaScript Connectors			
	<concise purpose>	Conditional connectors should be named succinctly to indicate their purpose. In the absence of a name, the condition will be displayed on the connection which is generally not readable and will be truncated in most cases	Is Allowed Message Process Message Cerner ADT only Is Error Condition Message for GERIS

Definitions			
Mapping Files	<source format>_to_<target format>	Describe details of mapping from one format to another	Cerner ADT HL7v23 to CareFusion ADT HL7v22.mdf
EDI Definitions	<source /target systems or tenancy>_<HL7 version / other standards>_<optional Other Information>	An EDI definition file may contain the message structural definition in use for one or more systems (particularly where Common Message Model formats are used) the naming should closely reflect the systems/tenancy/purpose the definition is used for and what standards are used, other useful descriptive information may also be included.	DHHS_OCIO HL7v23 Baseline.s3d GERIS_ALF HL7v23.s3d SHARED_UFD CMM v24.s3d INT-21 CSV_REFv23.s3d
Message Properties			
Local Properties	<concise name>	Solution properties and Transient properties (only relevant to the set of route stages that populate and use the property) should be named appropriately to indicate the value it stores. In some cases, properties populated during route processing may require to be prefixed to aid in easy identification of their purpose and the solution that	sendingApp sendingFac patientUR

		delivered them – this is especially true of properties that are expected to be populated by a publishing route and passed to any subscribers who require them for downstream processing (typically these properties should be managed within the configuration documentation to support easy identification and use by new subscribers)	
	As few as reasonably possible	Only populate the minimum set of properties required to support indexed properties searching or where the extraction / calculation of values from the message would produce a different result and the original field value is required downstream. Use of excessive properties will result in additional storage use by the solution.	
	Indexed Properties	Set and index the group of properties that are typically used as search parameters in “message contains” searches to improve the efficiency of Message searches / reduce the impact to disk I/O.	SendingApp SendingFac ReceivingApp ReceivingFac MsgDTTM

		These should be input indexed on a route that receives messages and output indexed on a route that sends messages.	MessageType TriggerEvent MsgControlID PatientURN EpisodeID Rhapsody_GUID VisitID PatientClass Surname Firstname OrderID AccessionNumber
Rhapsody Variables			
Inclusion	<external system> +/- <type / use >	Names of Rhapsody variables should be meaningful and convey their purpose in an obvious manner. Where applicable prefix with the external system name, and include information of the type or use of the variable (typically as a suffix)	AH_PAS_ADT_IN_PORT PAS_DB_HOST PAS_DB_PORT PAS_DB_DSN PAS_DB_USER PAS_DB_PWD MYHR_LIS_DELAY_ACTIVE ENV_PROCESSING_ID DEBUG_ACTIVE

3.5 Security and Access Controls

Most Rhapsody Communication Points support secure transport and/or authentication protocol. Rhapsody Health's recommendation is to always enable the secure transport options, providing the upstream or downstream systems also support this.

In addition, where authentication is required (e.g. basic authentication, WSS for SOAP etc.) our recommendation is to always opt for encrypted/hashed passwords. This mitigates the risk of plaintext credentials being captured in message log as part of transported message.

It is also the best practise to keep the security setting identical between the production and at least one non-production environments.

3.6 Route Design

3.6.1 Discard Before Processing

In general, conditional filtering of messages should occur as early as possible to ensure that additional processing of the message is not incurred (particularly expensive operations such as transformation, output mapping, JS filter operations). There are various methods to do so, for example:

- Conditional Connector
- Conditional logic within filters which support it (e.g., JS Filter)

3.6.2 Fail-safe Fail-early

Never assume that data sent to a route will always match the specified format. Always ensure that if a given route requires data in a certain format that it is verified within that route before further processing and invalid data is handled specifically (e.g. discarded via no match connector or passed to an error handling process as may be required depending on the route function).

Error detection should occur as early as practical so that messages are trapped or discarded before additional processing steps are incurred. For example, if a given type of message will always be discarded for a given output ensure that this is detected in the early stages of the output sequence / logic, rather than in the later stages.

3.6.3 Intra-Route Error Handling and Exception Processing

Error handling covers a wide range of topics, but the goal is to create a configuration where errors are the exception. Error handling should be actively planned and included as a part of the interface design.

Route Error Handlers may be added, as appropriate, to each route to handle edge cases / exceptions, and can also be used to direct known errors to appropriate destinations such as email notifications or a sink. The goal should be minimising the number of errors that go to the Error Queue. For production interfaces the most likely stage for encountering errors will be the input/receiving routes, use of route error handlers for processing or output routes will depend on the solution design and blocking behaviour requirements.

If the Error Queue is left to grow over time without careful management, then it will reduce the data volume available for the live message store – in severe cases this can contribute to running out of available data store disk space earlier than calculated or expected.

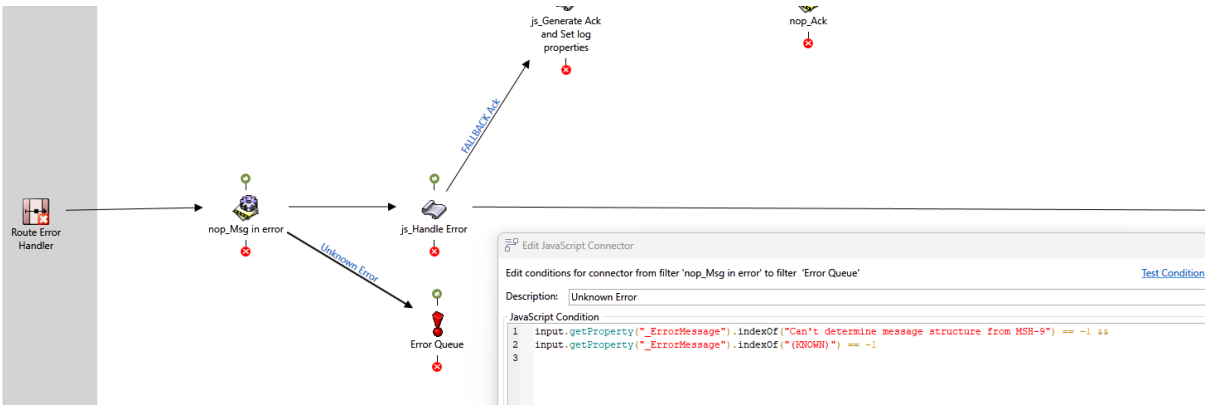
In-Filter processing can also be used to detect and handle certain error conditions such as:

- Missing required information / fields
- Logical error in a combination of fields
- Discrepancies in the expected message format or type compared to expected/known messages
- Messages which are not valid based on business rules

JS Filters may be used to call the addErrors() method to add specific information about the error and can then cause the message to be output via the error connector and routed as appropriate (e.g. to route messages to trigger an email when coded values are detected in fields from a certain system); or if appropriate directed to the Error Queue.

- **NB** Care must be taken not to use the pattern of a route error handler plus a JS filter adding errors excessively. The use of a Route Error Handler is only useful in scenarios where some known issues should be handled directly in route and others may need to be raised by throwing those errors again
    - **The downside to using a Route Error Handler + JS Filter** is that the user cannot simply click 'Reprocess" to have the message re-processed at the original error filter, because reprocess will simply send it to the JS filter connected to the Route Error Handler. Therefore **DO NOT** use the pattern where the filter after the Route Error handler simply re-throws the error with essentially the same information and no other processing. An example of appropriate use is:
        - A sending system is known to occasionally send a malformed message with a structure or type that cannot be parsed, this message causes the normal processing filter of the input route to throw an error
        - As shown in the figure below the Route Error Handler splits after the no op and string-based logic is used to create an ACK/NACK for the sending system. Additionally if the conditions of the connector are met then the message is also router to the error queue (in this case that the error is caused by something other than the known malformed messages with incorrect data in MSH-9). If it is the known condition no additional error is required / produced

Figure 10: **Example Appropriate Use of Route Error Handler**



The method for handling the errors will be determined based on the interface pattern and requirements. Error Handling should be designed and implemented (as per pattern and customer requirements) and adopted during the initial configuration rather than being added at the end.

Methods for handling errors include:

- Unhandled: the message is sent to the Error Queue (this should be the last resort, not the norm).
- Direct handling: logic is implemented to manage messages with known errors; the error detail is available from the message object by calling the getErrors() method in a JavaScript filter. Once processing is complete, the message may then be:
  - Ignored, as this is a known error which does not require further processing, and output to a Sink
  - Handled explicitly by output to a Communication Point/Route for this purpose (e.g. to automate raising a helpdesk ticket or to email specific people with the event details)
  - Re-routed to the error queue.

Note that errors relating to filter failure (rather than processing of the message content) bypass the Error Output connection and are placed directly into the Error Queue. For example, system-level errors such as Out-Of-Memory errors, will result in this behaviour.

3.6.4 Route Layout and Processing Complexity

In essence, the route design canvas of the IDE presents a map, or a flow chart of the high-level logic required to process messages. The following are recommended to ensure that the flow of data through a route can be easily interpreted and checked to ensure that the route logic is consistent with the requirements as well as for streamlining support and maintenance.

- Message path flows generally from left to right (input to output).
  - For some scenarios, the route may output then return to an input path such as when processing a message and then the Acknowledgement via an Out -> In communication point. This is a normal scenario, and the flow is still generalised as left to right, like the operation of a cursor on a page of typing
- Input connections should be made from a component to the left of the filter
- Output connections should be made to components to the right of the filter (this also includes No-match and Error connection points)
- No-match connections should generally be made above and to the right of the filter

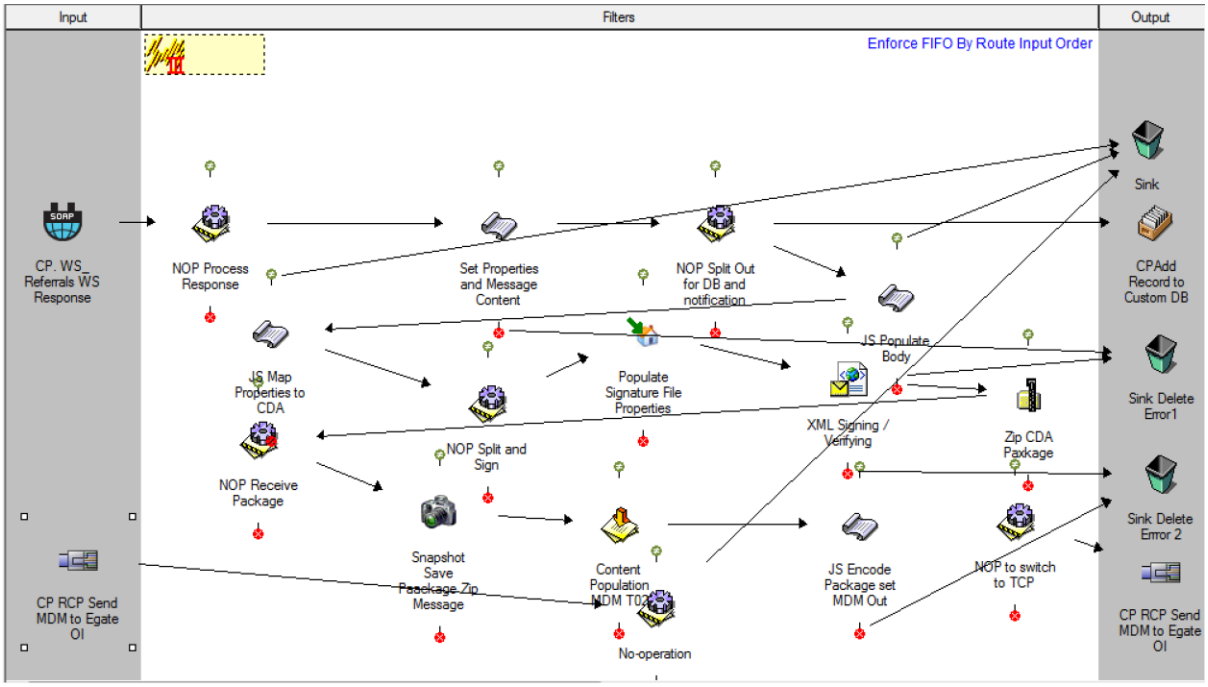
- Error connections should generally be made below and to the right of the filter
  - Note that the use of the Route Error Handler may streamline error processing and avoid requiring individual paths from Error Connectors, as well as No-Match Connectors in some scenarios (if a condition is not met and there is no path from the No-Match the message will route to the error path)
- Connector paths do not cross each other, where possible
- Avoid loops within individual filter paths that bring the message back to an earlier point in the same filter pane of an individual route
  - Note this is different to a multi-stage interface where a later stage route may need to end back to an earlier route for further processing based on current state of data, responses received from downstream systems etc.

Overly complex routes with multiple significant branching and/or return to filter components generally indicate that the configuration is attempting to achieve too much in a single route (Figure 11). In this case, consider splitting the single route into several route stages, each of which performs a specific function before passing the message to the next stage for processing (Figure 12 - Figure 14). This can be achieved either using Dynamic Routers or through the method of Linking/Chaining Routes (see below).

In addition, this type of configuration is often the result of adding later business logic steps that want to reuse the already-tested logic from an earlier step in the process. Use of shared JS Libraries in more recent Rhapsody versions to contain that logic allows use of the logic in multiple steps and helps avoid this type of reflection in some cases.

The below figure shows an example of a monolithic route that would be better deployed as a series of route stages to simplify the overall layout and avoid potential logic issues:

Figure 11: **Monolithic Route**



The below three figures show an example of a simplified layout which target smaller, more specific, areas of functionality using several route stages:

Figure 12: **Simplified Route Layout – Stage 1**

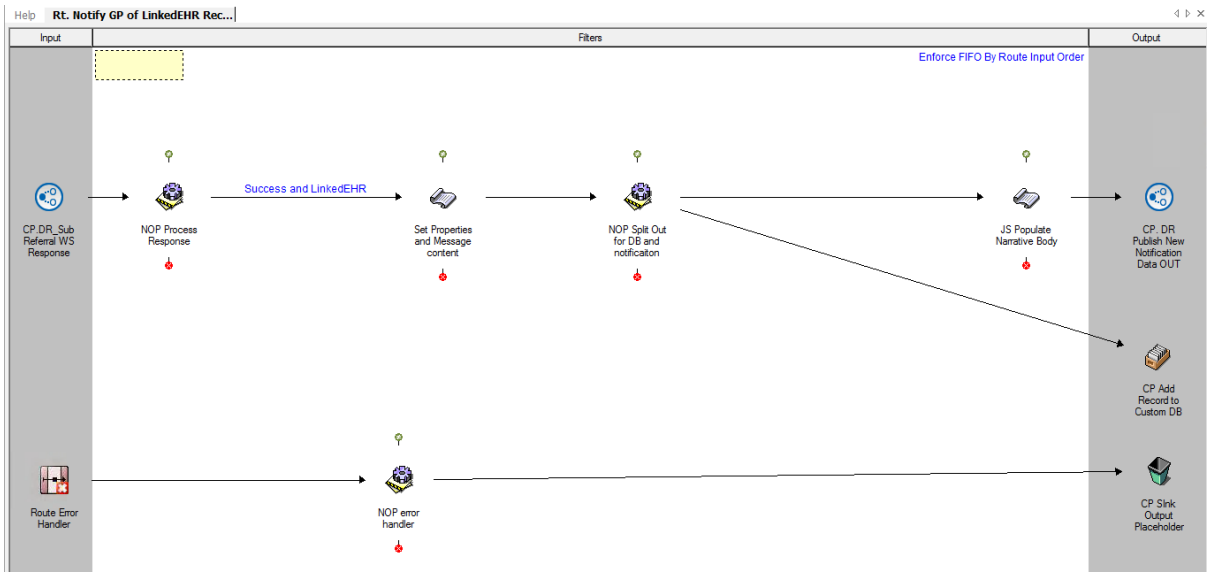


Figure 13: **Simplified Route Layout – Stage 2**

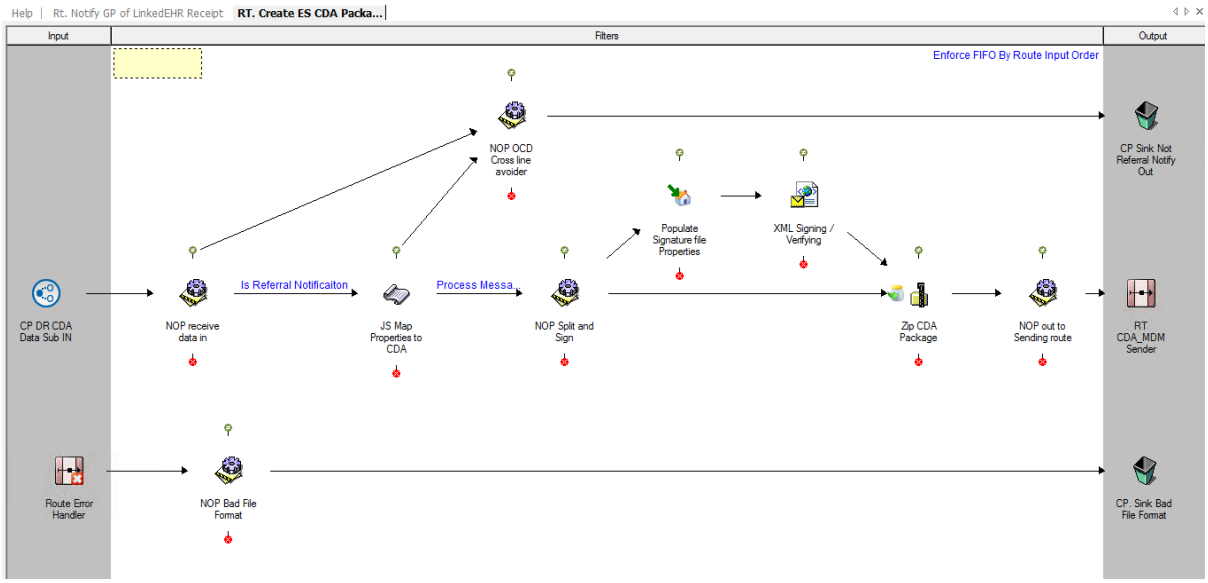
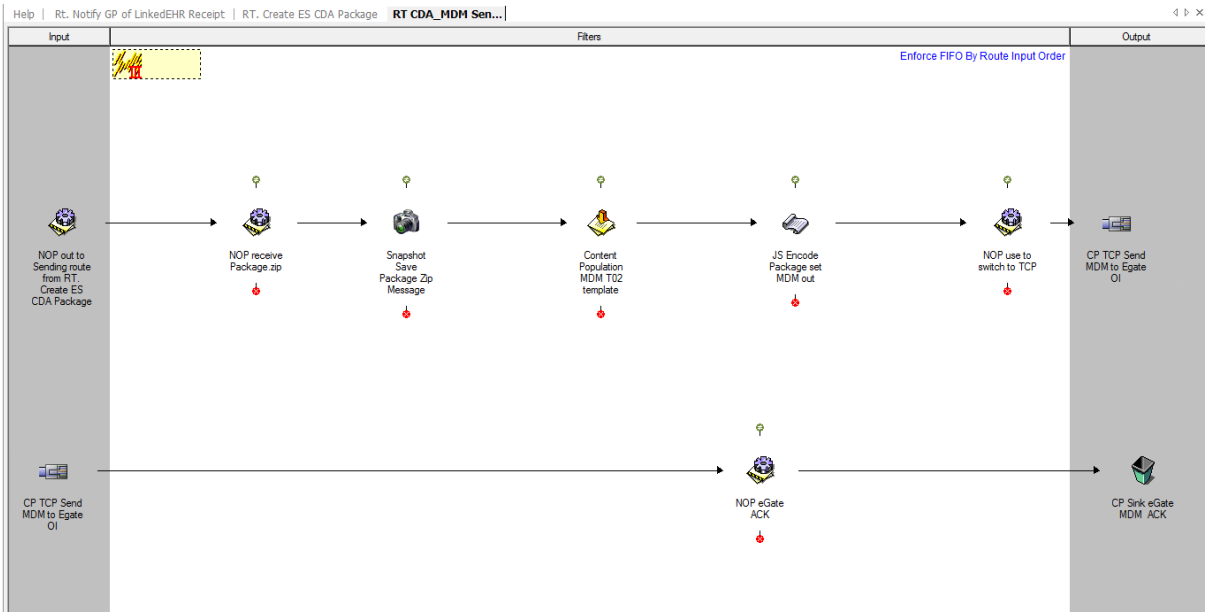


Figure 14: **Simplified Route Layout – Stage 3**



Note the preceding figures aim to demonstrate a solution for simplifying the original route with multiple reflections backwards in processing, in terms of best practice these could be further improved by collapsing multiple filter steps into single filters where possible. For example, in Stage 3 the function of the content population filter could be encapsulated in the filter "JS Encode Package set MDM out".

3.6.5 Linking/Chaining

In a complex configuration, it is frequently useful to separate the processing logic onto multiple routes, limiting each route to one high level function and to ensure that the clarity and purpose of each route is maintained. Note that this conflicts with the best practice principle of using as few filter steps as possible in order to reduce the number of disk-writes to the datastore and should only be used when necessary / where it is not possible to achieve the outcome with reduced numbers of filter /processing steps.

In some scenarios, it may be beneficial to directly link the routes in a chain (this is generally the case where the overall requirement cannot be achieved unless all the links in the chain execute in sequence) – for other cases we would recommend the use of dynamic routers to separate the stages of the route processing.

When routes are chained, there is a risk that re-arrangement of filters or changes to message paths may lead to disconnection of the link. For this reason, when chaining routes together follow the following process to ensure the connections are managed properly and can easily be identified:

- Place a no-operation filter as the last processing element in the up-stream route.
- Place a no-operation filter as the first processing element in the downstream route.
- Drag the downstream route (selected from the configuration workspace) onto the output side of the up-stream route.
- Connect the no-operation filter on the upstream route to the route icon in the output frame. This will place the connection into the correct location on the paired route.
- Connect the no-operation filter on the downstream route to the connection icon in the Input frame.
- Check in both routes to save the configuration.

This model is often described as providing book-ends to the routes, and generally protects the connections between the routes. It is, however, prudent to confirm the connectivity between the routes if changes are made to either route.

Figure 15: **Bookend Routes – Route 1**

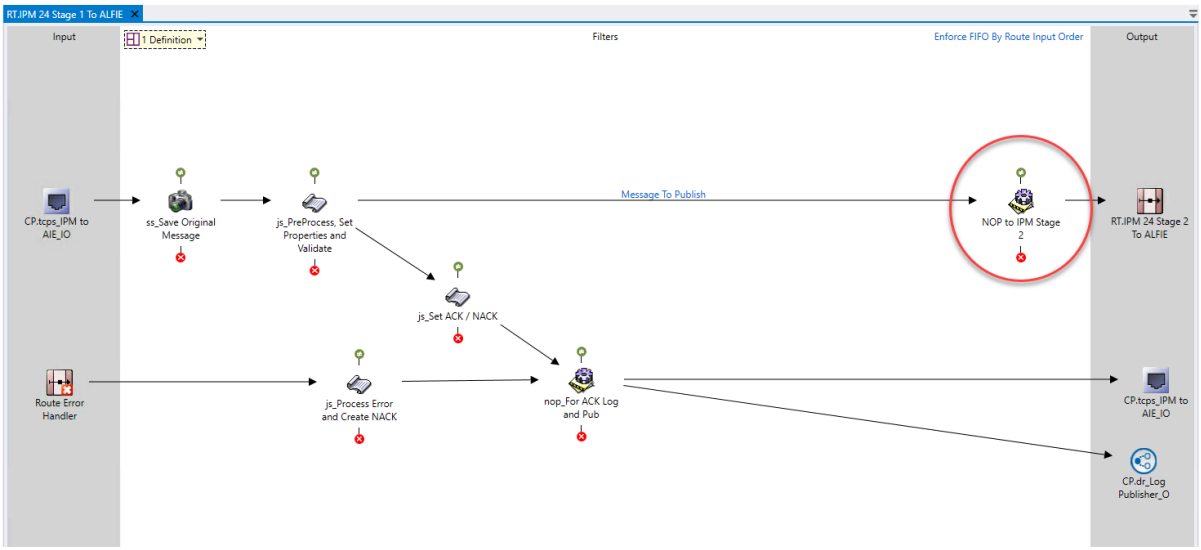
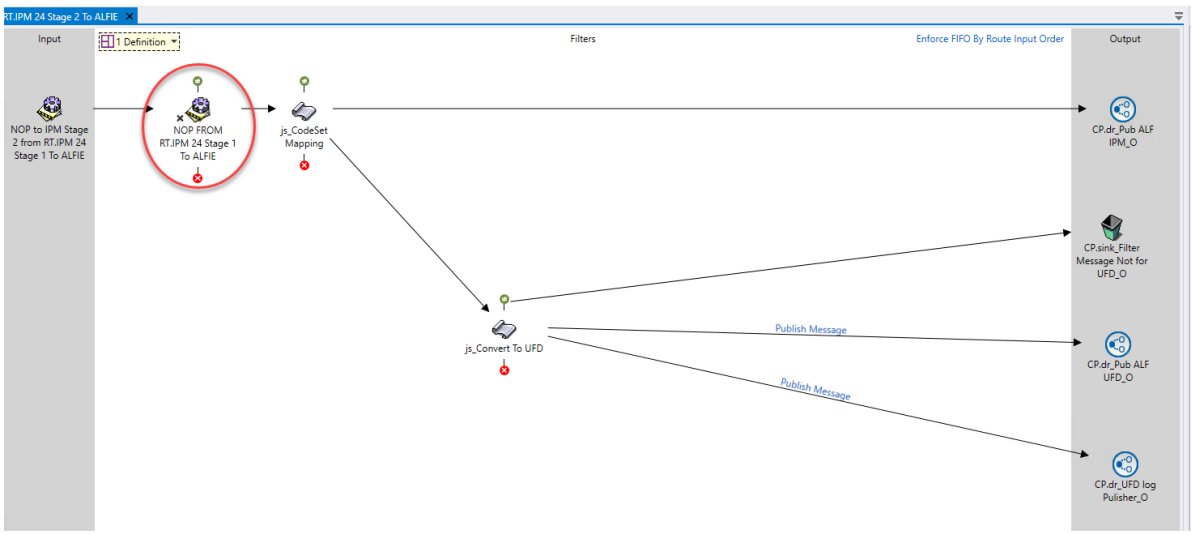


Figure 16: **Bookend Routes – Route 2**



Note that multiple input connections may be made to the output route icon in the up-stream route. Each will result in a separate input connection to the downstream route, thereby allowing separation of processing for each path.

3.6.6 Message Definitions and Parsing

If a message definition is placed on the route, message parsing will occur prior to processing by the first filter on the route. This ensures that properties defined at route level (such as the standard Input Message Type, as well as properties

populated by extraction) are available to conditional connectors and certain filters.

Message parsing is required by the following communication points and filters. The process costs time and resources of Rhapsody, therefore, it should only be undertaken as necessary.

- Acknowledgement Generation
- EDI Message Validation
- Content Population
- Code Validation
- Date Validation
- Mapping filter
- Code Translation
- JavaScript filter (if the message has not previously been parsed, and the logic is using a method which requires parsing)
- XSD Validator
- HIPAA filters
- Database filters and communication points that use message fields in the query
- Freemarker Mapper filter that use edi message fields in the mapping template

Use the following guidelines to avoid unnecessary parsing / re-parsing of the message:

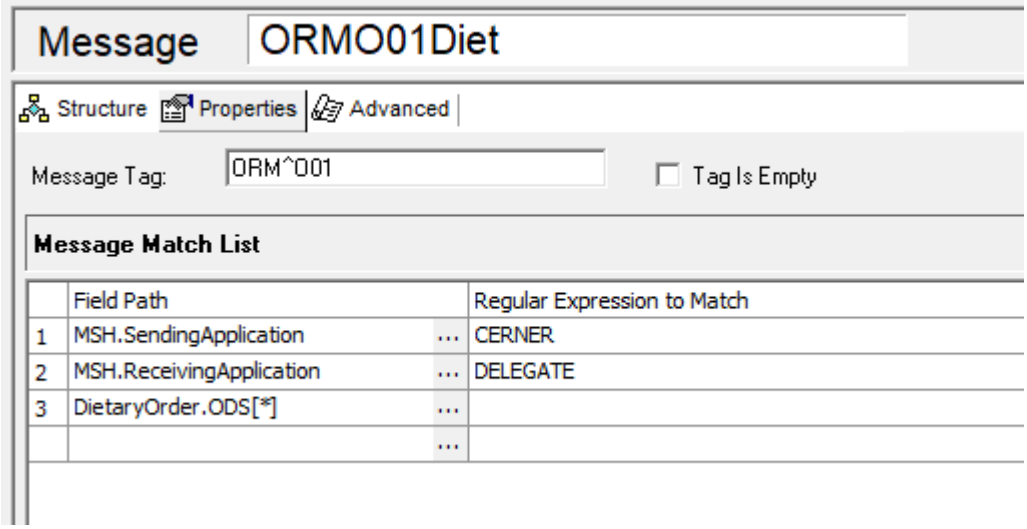
1. Place only one message definition on a route if parsing is required for the route to operate
2. A message definition is not required for the following scenarios:
    - Pass-through routes
    - Routes with messages that are processed purely using the following methods:
        - String based processing
        - XML Processing via E4X
        - XPath processing of well-formed XML
        - HAPI message processing (as HAPI does not rely on the edi parsing engine)
        - HL7 Message Modifier filters
        - JSON Processing

        - Certain scenarios using Web Services or Database Orchestration
3. Avoid using multiple filters which require message parsing when possible (more generally aim to use as few filters as possible)

   JavaScript filters that cause a message parse operation for each filter – where possible combine multiple JS filters into a single filter (the use of Shared JS Libraries can be used to offset the overall complexity and maintenance effort for single JS filters – see the JS Filter best practice section)
   - Message Validation filters on a route with JS or Mapper filters - often the validation can be achieved within the other filter type allowing the removal of a parse (and potential data store write) operation e.g. via use of the edi object validate() method.
   - Property Population filters which map message fields to properties on a route with JS or Mapper filters – again the JS/Mapper filter can typically achieve all the required functionality
   - Ack Generator Filters – can be replaced by string processing / HAPI processing within a JS Filter and combined with the other business logic operations
   - Content Population filters can be replaced with string logic processing within JS Filters
   - Mapping filters can often be replaced by equivalent functions in JS Filters, even where format changes are required this can often be achieved via supplemental string processing in the JS filter. Typically, most routes that have a Mapper filter will also have a JS filter (as there are features/functions available within JS filters that are not available to the Mapper) and combining these where possible reduces the parsing requirements, the data store use, and generally improves maintainability
   - Similarly targeted filters such as the date validation filter can be completely replaced via JavaScript libraries executed within JS filters that perform the equivalent function
4. Avoid adding multiple message definitions to a single route. Rhapsody does not guarantee which message definition will be used in this scenario. The first definition which allows a successful parse may not necessarily be the expected one, which can lead to inconsistent results. Typically, if a design pattern indicates more than one definition is required on a route to achieve the outcome then it is probable that:
   - The solution is performing a mapping operation that is better handled through either the JS Filter with supplemental string processing, or via the mapper filter

- The route is attempting to manage messages from multiple systems which have different structural formats within a single route. The following approaches can be taken to avoid this:
  - Create multiple message sets (or message sets with multiple messages of different structure) to handle the different formats within a single s3d definition file so that only one definition file is required. In this scenario you need to consider that Rhapsody will traverse from top to bottom and use the first message that can match to parse, thus more specific matches should appear higher in the order and should use the additional Message Match List criteria to correctly identify the message structure to be used for parsing such as the example below which will match an ORM^O01 message from Cerner for Delegate and which contains repeats of DietaryOrder.ODS segments:

Figure 17: **Example Message Match List**



  - Use multiple routes with separate definitions to handle the different message types/structures
  - Split the route into multiple stages, each targeting a specific area of functionality and only requiring at most one definition (potentially this pattern could be used to convert the different message formats into a common message model format)

3.6.7 Multi vs. Single Threading

Rhapsody is a multi-threaded Java application and uses 10 worker threads (by default) to manage all the route processing. In general, the processing cycle is similar to the following:

- Route worker thread becomes available and is in the "waiting for message" state
- Thread pulls the next available message from an input queue
- Thread processes the message from input to output (e.g. Property Population, Filters such as DB Lookup, JS etc.)
- Processing on the last route filter completes and the message is delivered to the output (Communication Point or Linked Route)
- Route execution finishes
- Route worker thread becomes available

Note – the preceding description is simplified and does not address workflows where the message may be placed on a message collector, which will release the Route thread once the message is on the collector; and the message(s) will then be subsequently released after conditions for release are met, then processing by a route executor thread will continue.

This design means that it is possible for multiple messages on a route to complete processing by a filter at more or less the same time, unless either filter concurrency restricts processing, call or the route is constrained to processing in strict message order by FIFO constraints (single threaded).

The filter concurrency is normally set to zero which allows the multiple and potentially out of order processing (note however when FIFO is set on a route even though the messages may be processed out of order due to different paths or complexity of messages in a given route, the messages will always leave the route in the FIFO order received). It may be necessary to limit concurrency in one of the following cases:

- Message processing order is significant; messages must process in the order of receipt (particularly if there is an in-route orchestration, and the processing result of a later message would be different if the earlier message has not been processed).

- Slow filter processing: in these cases, under moderate to high loads, it may be possible for all of the worker threads to be processing messages through an instance of the same filter at the same time which will effectively block other processing.

Filter Concurrency is available as a filter property for filter types likely to experience this issue. Setting the concurrency to a non-zero value limits the number of messages able to be processed through the filter instance to that value. The Database and Execute Process filters are particularly prone to needing attention to concurrency.

3.6.8 Special consideration for use of dynamic routers

Use of Dynamic Routers is recommended to decouple routes which publish/distribute data, process data or subscribe to/receive and send data to downstream systems; earlier versions of Rhapsody were constrained to the use of chained routes to achieve the same outcome. Use of Dynamic Routers at the boundaries of internal routes are advantageous because new consumers of existing data can be added in the future without needing to modify existing, validated, in-production configuration, thus mitigating the risk of introducing issues or regressions to current function.

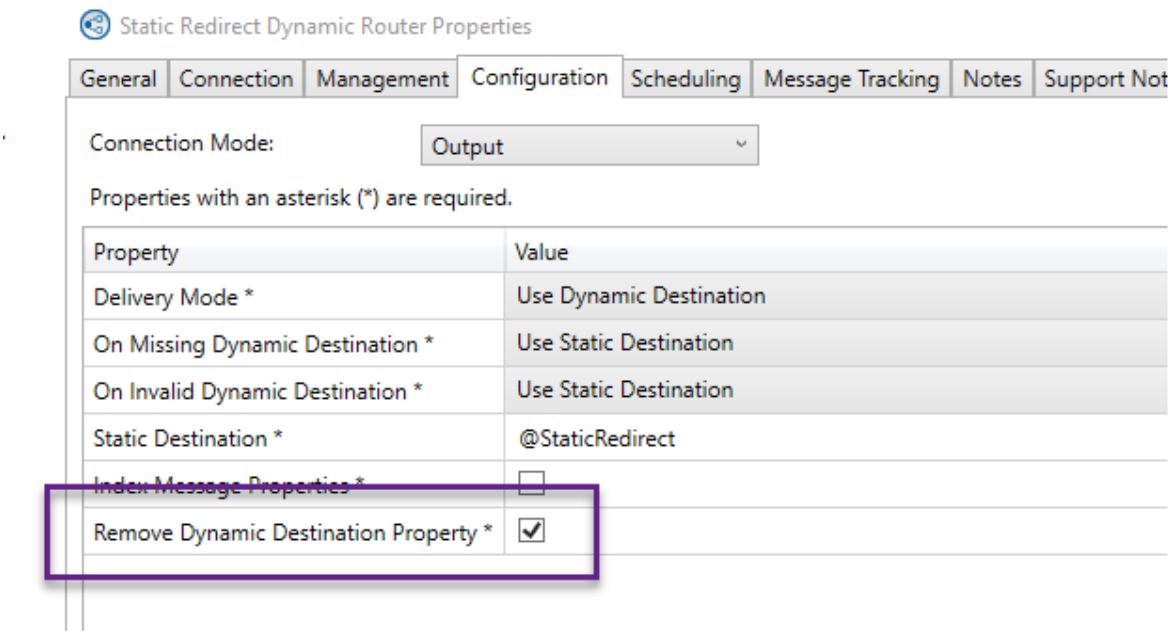
However, because data is Published/Subscribed to in a dynamic fashion there is potential to introduce issues within configuration if the following guidance is not followed:

- Subscribers (routes which use a Dynamic Router (DR) configured for input mode)
  - The route which contains an input DR should verify the data is of required type before performing any operations. This prevents errors or exceptions if data sent to the input DR has changed after deployment or is incorrectly directed to the component.

    For example, a source system could start sending new data types via an existing input interface; or a newly deployed configuration could match an existing target name of an input DR.
  - If present the dynamic router:Destination property should have the current value removed (set the property value to null if it is not required, or to a specific value as required) to ensure that processing through any down-

stream output Dynamic Routers does not result in route loops. Note that recent versions of Rhapsody now include the capability to remove this property automatically as part of the function of the Output Dynamic router via the selectable option shown below, this is selected by default:

Figure 18: **Remove Dynamic Destination Property**



- Publishing routes (routes which use a DR configured for output mode and may use dynamic destination lists set programmatically via in route JS processing)
  - Use Target Name (e.g., @theTarget) if possible, multiple receivers can be configured with the same target name to receive the set of data from the output DR
    - This method is simpler to manage and less prone to error than the path method. It supports delivery to multiple components with the same target name. In addition, the path value would have to be updated if the component is moved within the IDE to a different path.
    - Use of Unique Target Names (by selecting the option shown in the image below) forces re-use of the same input Dynamic Router in subscribing routes. This can be advantages in identifying all the routes which process data from the same source / publisher via the "show uses" feature

Figure 19: **Unique Target Name**

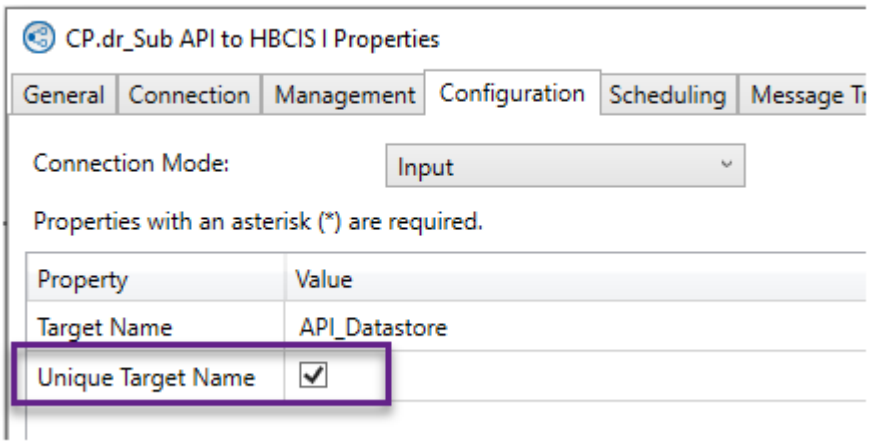
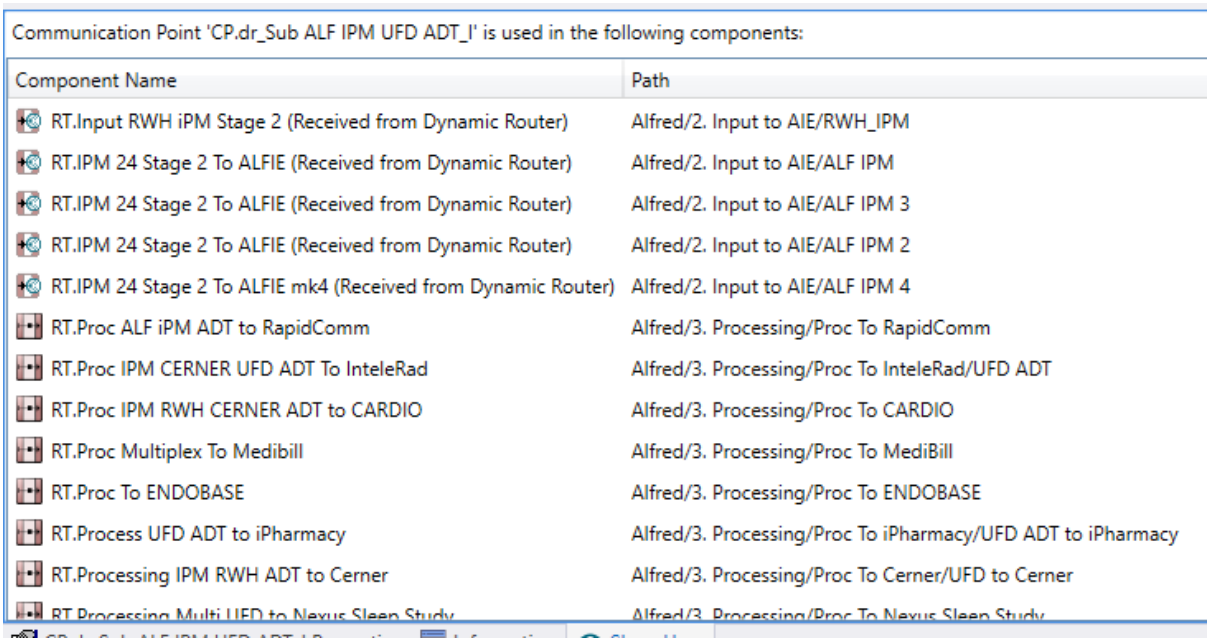


Figure 20: **Show Uses**



- While a single instance of an output DR communication point, using dynamic destinations, can be used in multiple routes to reduce communication point clutter, there are disadvantages to this approach which need to be balanced for a given solution's requirements:
  - The Archive queue will contain data sets from the multiple routes - this may obfuscate the cause of issues or increase overhead for some Management Console admin tasks.
  - If the communication point must be shut down for any reason it will

block the output (and result in queuing) for all routes that this output component is contained in (there are scenarios where this may be required e.g. to support message queuing while updating the downstream route).

- Where destinations are set dynamically, care must be taken if the list of destinations being set within the JS filter is populated from a source which can change (e.g. a Lookup Table). If a target name does not exist, the message will be routed as per the configuration setting for "On Missing Dynamic Destination".
  - We recommend immediately adding an input DR with a matching Target Name and checking in the component whenever making changes to dynamically generated destination lists.
  - Where retiring a component, remove the destination from the generated property list / lookup table first then retire the matching input DR component/route.

3.7 Queue Depth

Communication points (depending on type) have an input and/or an output queue (depending on their directionality) while routes have a processing queue. In general, messages flow through the Rhapsody engine mostly unhindered. Messages may queue on some components at some stages because of load, slow processing, or some external factors, but queuing is generally transient.

Factors which can impact the size of queues include:

- A requirement to throttle messages or only send messages to downstream systems during certain hours
- An issue at the receiving system end that will result in a backlog within Rhapsody until the issue is resolved
- A system which sends messages in large batches or bursts (either due to the nature of the information exchange or because it has been offline and needs to send a large amount of data that would normally flow over a longer period at a lower rate of messages per minute)
- A system is decommissioned while messages are still being processed within Rhapsody

- Large numbers of messages spawned from a de-batching process or where single input messages produce large numbers of output messages
    - Where possible messages should be debatched on a communication point as this has a much lower impact on use of compute resources, and each debatched message will have a separate original message identifier
    - If in-route spawning / debatching is required (e.g., create a set of REF documents from rows in a CSV file) be aware that each spawned message will still be linked to the original message. When these messages are sent via a Dynamic Router in output mode if the number of messages is greater than the limit for Infinite Loop Detection these may be directed to the error queue. This can be avoided by either:
        - Increasing the value of `RouterModuleService.maxRouterSendsPerMessage` in rhapsody.properties (the default is 50) – note though that this will apply to all messages engine-wide
        - Setting this property on the debatched/spawned messages so only those are specifically excluded: `router:SuppressRouterInfiniteLoopDetection`
- Resource intensive message processing and/or very large message processing within a route
- Single threaded/Strict FIFO requirements

Attention should be paid during route development to the potential for messages to queue (particularly with respect to the last two list items above which are controlled by Rhapsody). To reduce the potential for queueing:

- Develop a solution using standard product-supplied Rhapsody Communication Points and Filters in preference to developing custom components. This reuses code libraries as typically the product-supplied components align with, and are tested with respect to, Rhapsody performance and multi-threading capabilities (which in turn generally avoid large processing queues in most scenarios).
- Constrain the use of Strict FIFO or interfaces with filter processing that requires no concurrency to only those use cases which absolutely require it to ensure minimal impact to engine performance and queue depth.
- Ensure components are correctly configured with Auto Start parameters to ensure queues do not form because one of the components has not started,

following an engine restart.

There are several significant impacts of queuing on the engine:

- Queues are managed as memory objects; consequently, large queues will lock memory for the queue and make it unavailable for normal processing.
- When an engine restarts, the active objects are validated before processing can resume; consequently, large queues will take finite time to validate and delay the resumption of normal processing by the engine.
- Messages on queues are not eligible for removal from the data store by the Rhapsody Archive Clean-up process (for the scenario of a decommissioned system the messages may need to be manually deleted from the queue, so they are eligible for clean-up).

3.8 Communication Point Considerations

- Wherever possible use an instance of a communication point on a single route only. This provides advantages in terms of managing blocking behaviour, queue management and message tracking.
    - There may be limitations and considerations for a specific solution (such as the licenced number of communication points) that need to be considered.
    - In some cases, it is advantageous to reuse a communication point on multiple routes to reduce maintenance overhead and effort for those communication points with significant configuration components – web service clients connecting to the same service with a number of different methods are an example of this. Database communication points used to log/store all received messages may be another.
- Use multiple sinks rather than a single, general-purpose sink. This allows you to see how many messages are being discarded for a particular system, which may in turn simplify identifying issues with discard logic.
    - Note communication points such as Sinks and Dynamic Routers where messages do not enter or leave the engine instance do not count towards the licensed number of communication points.

- De-batch large messages in a Directory communication point instead of a de-batch filter (either the Batch/De-batch or Zip/Unzip filter). This is more efficient as the communication point can read each message out of the batch from disk without loading the complete batch into memory, and as noted in 2.6.8 Special consideration for use of dynamic routers this has benefits in terms of infinite loop detection
- Ensure the retries, "Connection Mode" and "Idle Timeout" properties for a communication point are set appropriately for the system(s) that it is connecting to; and these have been tested to ensure it is tuned appropriately.
- Environments where firewalls or network load balancers sever idle connections without completing the correct socket close sequence server style: Communication Points may stop on error due to the number of retries being exceeded. If this is encountered the number of retries for the server should be set to infinite
- Where multiple separate actions need to be performed on a message when it is received by a given route, it should be received on the route then split via the use of a No-Operation filter for each action. Do not directly split the input from the input communication point as this results in duplicate parsing of the message and duplicate storage of the input message and properties within the Rhapsody data store as shown below:

Figure 21: **Correct approach for message splitting**

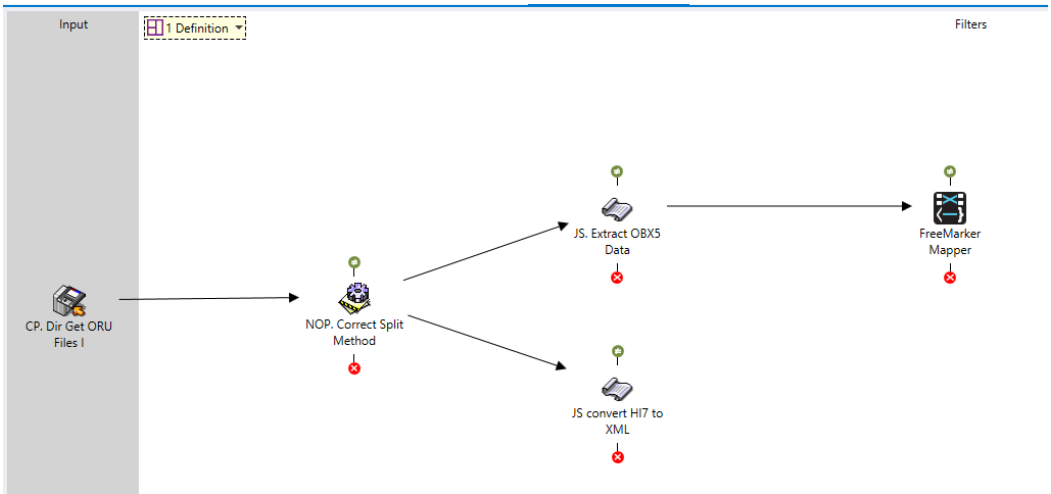
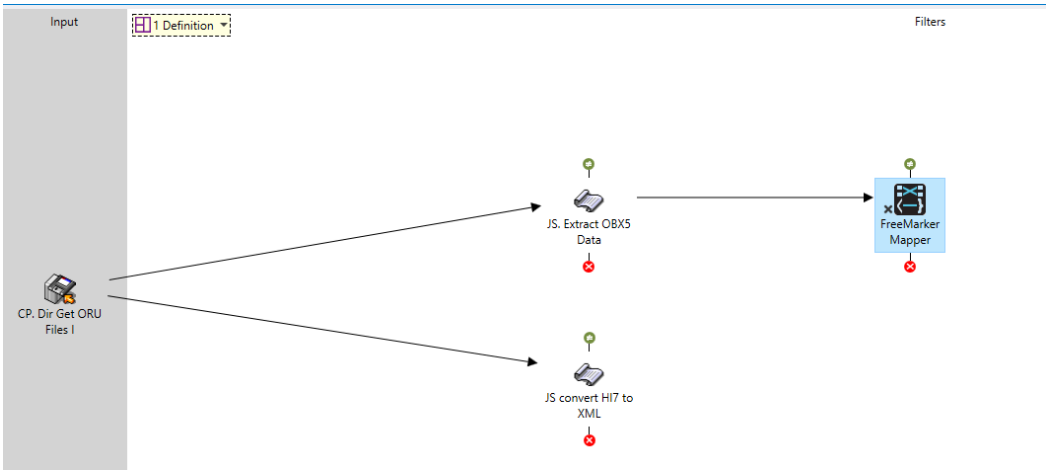


Figure 22: **Incorrect approach for message splitting**



- Where connections between two Rhapsody instances is required, the Rhapsody Connector communication point can be used instead of the TCP/IP communication point. This communication point should be used where the current message properties are required by the downstream Rhapsody instance as they will be serialised along with the message.

3.8.1 Advanced Routing

From Rhapsody 6.5 Advanced (Continued) Routing on success / failure is available. Rhapsody Health recommends using Continued Routing on Failure to capture and manage known patterns rather than simply sending these messages to the error queue, for example where response timeouts are received and ignored.

3.8.2 HTTP Communication Points

HTTP communication points have a large number of properties passed as http headers which will impact the amount of storage used when updates are made to the meta store. Route processing of message workflows to / from these communication points should process the properties and remove those which are not required / no longer required to avoid downstream propagation of large amounts of data.

3.8.2.1 Other Known Limitations

3.8.2.1.1 Out-> In Round Trip Timings

Per internal Jiras :

- https://lyniatejira.atlassian.net/browse/REN-32837
- https://lyniatejira.atlassian.net/browse/RSR-178#icft=RSR-178
- https://rhapsodyjira.atlassian.net/browse/RSR-441

There is a limitation in the way the library (Jetty) used to provide the HTTP communication point reports message latency (the time taken for a response from the downstream system for a message sent by Rhapsody). Unlike other communication points the library does not return until the response is received, or the timeout is exceeded. This means the timestamps for "sent " and "received" are almost instantaneous and this makes the information displayed in the Management Console and the Message Latency Reports appear as though Rhapsody is queuing a message for long periods before the downstream system responds quickly whereas, in fact, Rhapsody is sending but then waiting a long time for the response.

- It is recommended when working with https communication points in synchronous workflows, or workflows that are timing / throughput sensitive to add properties with timestamps of when the message leaves the route to the comm point and when the message is placed back on the route from the received response in case these are needed to produce alternate reports to prove to a customer (or 3rd party who engages us) that the latency is caused by the downstream system's response time and not due to Rhapsody delays.

3.8.2.1.2 Connection Threads

Per Internal Jira:

- https://rhapsodyjira.atlassian.net/browse/RSR-837

The HTTP communication point is configured with a <limited> number of concurrent connections e.g. 25. In some scenarios badly behaving clients connecting to Rhapsody do not disconnect after a transaction leading to starvation of connections and new connections being rejected.

To avoid this scenario for HTTP server communication points ensure that the idle timeout is set to a reasonable value as this will allow Rhapsody to release the connection if no data is received from the client after that length of time.

Also be aware per Internal Jira :

- https://rhapsodyjira.atlassian.net/browse/RIE-1818

That in versions of Rhapsody prior to v 7.0 the number of simultaneous connections is limited by the default value for the Jetty Library which is 200 – even though you can configure more connections/threads in the UI this is because the general tab is shared by all components and does not take into consideration the limitations of the more specialised types.

From version 7.0 the default number of simultaneous connections (thread pool) can be modified by updating the rhapsody property

HttpModule.HttpServerCommunicationPoint.MaxListenerThreads

And restarting the server.

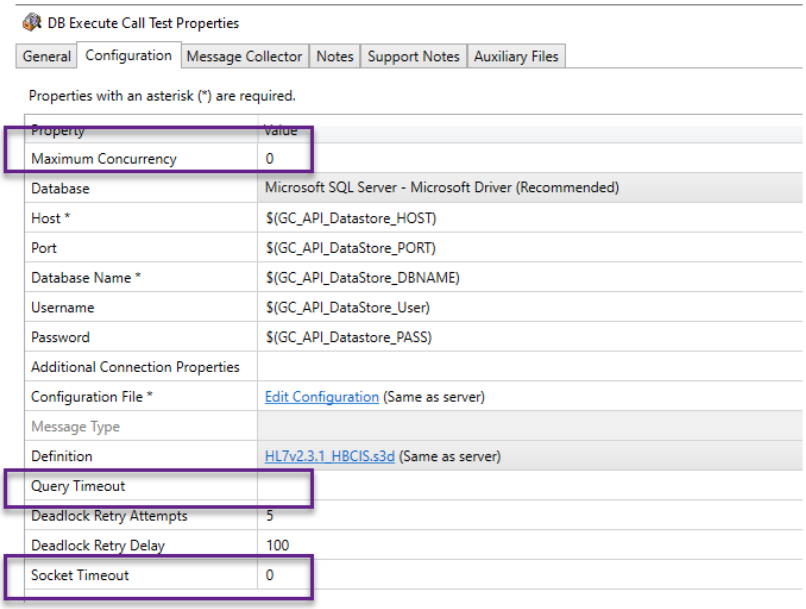
3.9 Filter Considerations

3.9.1 General Considerations

- Combine filter functions where possible to avoid multiple message parses and increased data store volume use (body and meta store areas).
- Only set filter concurrency to a non-zero number if performance analysis indicates the filter is consuming large amounts of resources and is negatively impacting overall system performance.
    - Setting filter concurrency to 1 will reduce the throughput (as the route will become single threaded for this filter step) and is only required in a small number of use cases (such as where absolute FIFO is required within the route, rather than simply for end-to-end message delivery).
- The use of database filters is only suited to certain scenarios as outlined in the Database Considerations section.
    - Where used be aware that the default configuration is not recommended as there are scenarios where downstream database issues lock all route processing threads and lead to halting of message processing. In

particular, ensure a non-zero value is specified for Query Timeout and Socket Timeout, it is also recommended to change the Maximum Concurrency to a non-zero value – this will require testing to balance against throughput requirements, however for most workloads it would be unusual to expect more than 2-3 simultaneous messages processing on the same route (this setting is not required for Single Threaded routes)

Figure 23: **Default settings which require modification for Database filters**



- Avoid using excessive No-Operation filters for route commenting – this practice introduces excessive growth of the events within the Message Store and has been demonstrated to negatively impact large scale deployments. No-Operation filters should only be used to
  - Split message processing paths
  - Improve route layout by avoiding crossed connectors
  - Support Message Collection (if this feature is used, ensure that the name of the filter reflects this)
  - Link to a chained route to avoid link loss when updating / removing other filters

3.9.2 Use of Conditional Connectors between Filters

3.9.2.1 Override the Condition Description

Always override the default condition description for the connector and replace it with the shortest text that makes sense to describe the condition (e.g. is Required Message Type, eReferral Only etc.).

3.9.2.2 Special Note on Use of Rhapsody Variables in Conditional Connectors

While our general rule is to use Rhapsody Variables for any value that may require update in future we do not recommend their use in Conditional Connectors for the following reasons:

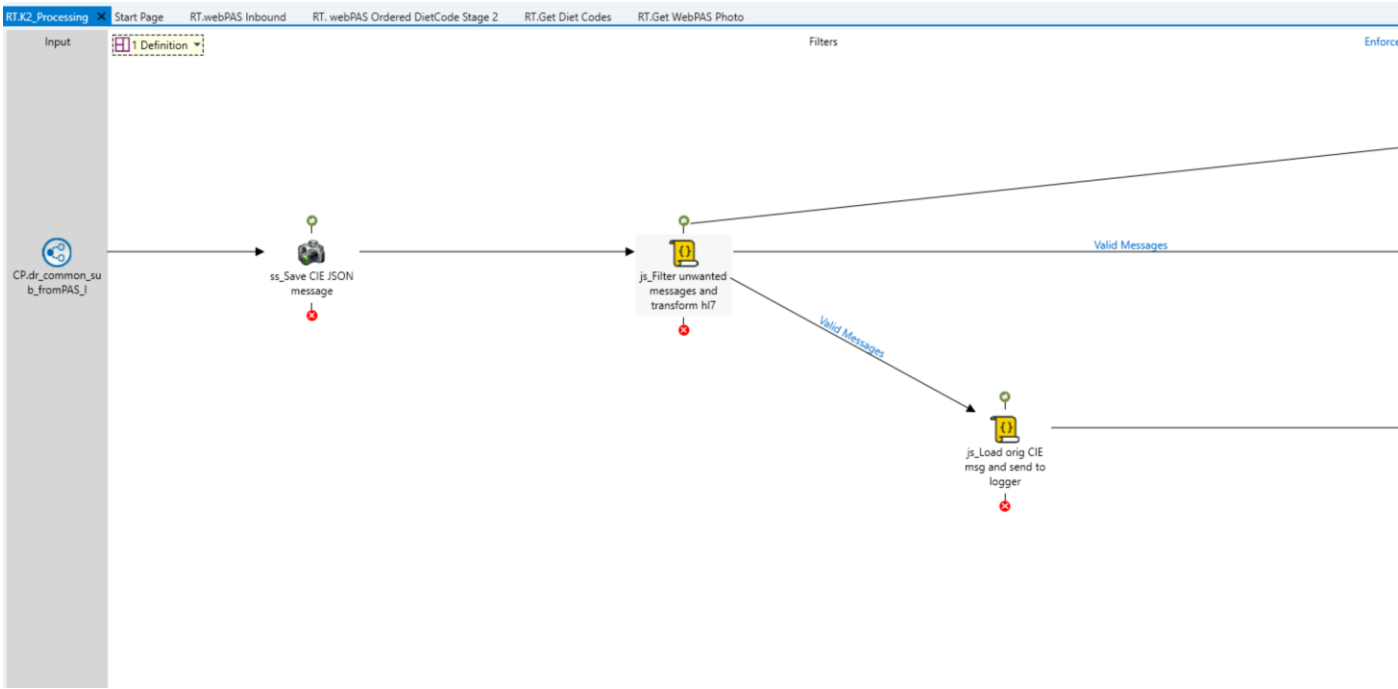
- The condition they are matching is not immediately obvious when viewing the component or troubleshooting condition test results – the user must access the Rhapsody Variables Manager which cannot be opened at the same time as the UI component of the Conditional Connector.
- In the current Rhapsody version (and all previous versions) they are not exported between environments unless you select the "Export Unused Rhapsody Variables" option – this can result in a lot of unnecessary clutter in the downstream environments which will have to be manually mediated.

3.9.2.3 Note on use of JavaScript Conditional Connectors

PSG – APAC do not recommend use of JS Conditional Connectors for the following reasons:

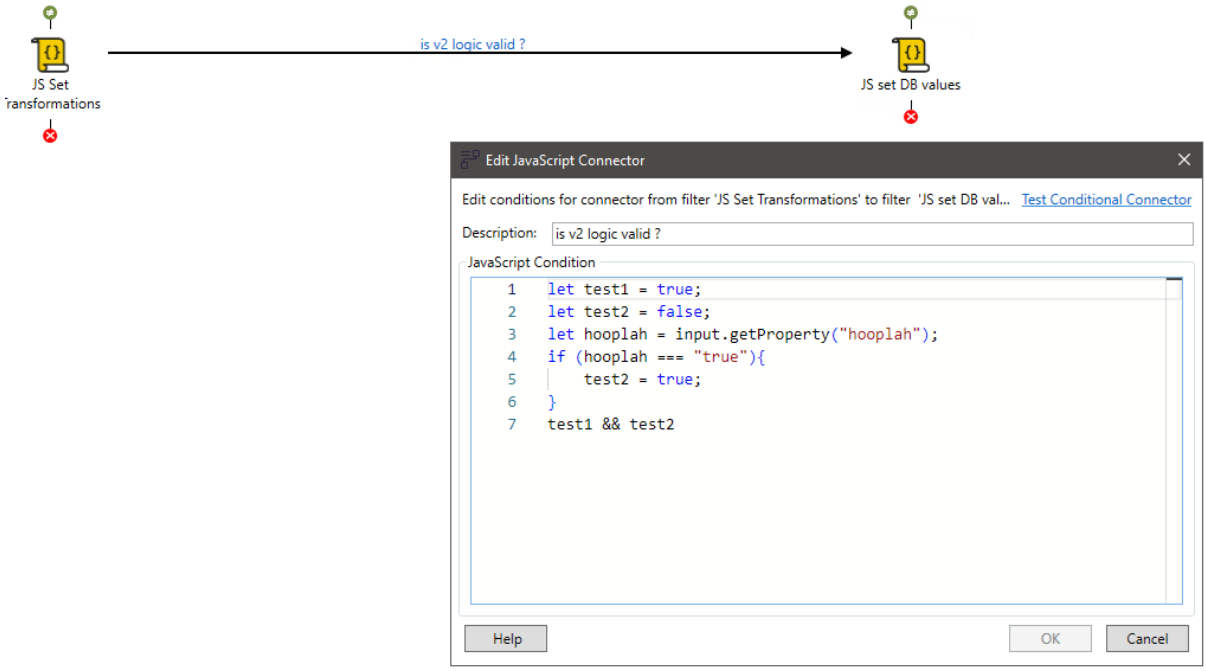
1. Any logic that could be performed for validity of the message being passed through the connector can generally be executed in a JS filter prior to the connector and used to set a property value that is used to allow / deny a given message through the path – this pattern has been commented on internally as being standard across the US and EMEA as well

Figure 24: **Typical Pattern for Conditional Control of Message Flow**



2. Use of JS connectors separates logic that should be captured along with other message operations – meaning users becoming familiar with , or needing to later maintain and support, a solution cannot inspect all the logic in a single place – and have to aggregate the information from multiple components
3. Future Upgrade considerations: Rhapsody 7.4 will introduce a JS v2 option for the connectors adding additional test and migration effort for customers who wish to move as much configuration as possible to the new libraries
    a. Testing shows in 7.4 the user is not able to choose between v1 and v1 unlike JS Filters and JS TCP Comm Points – be aware of this when asked to review/ tests to v7.4 from earlier versions. Even though use is low there may be some customers using these which have features not supported by GraaL

Figure 25: **JS Conditional Connector Screen in 7.4 – No Option to Select v1**



Note during upgrade investigations for later releases of Version 7, where the JS conditional connectors are to be converted to Graal, it was revealed that use of this feature is low /almost non-existent within Rhapsody Internal or by our customers.

3.10 JavaScript Use and Features

3.10.1 Code Guidelines

Below are the general guidelines for writing JavaScript code within Rhapsody:

- Do:
  - Use the Shared JavaScript Libraries to add functions that can be re-used across JavaScript filters.
  - Improve code readability and self-documentation through the long hand versions of constructs such as the ternary operator.
  - Code within the JS filter or the function within the JS Library should follow the same documentation convention as the route notes for capturing the date, author, purpose, and modification history.
  - A consistent coding style should be used for all filters and shared JS

libraries in a given solution – for example there are multiple valid methods for declaring variables, however it is simpler to understand and maintain code if a consistent approach is used

- Avoid:
  - Avoid using multiple JavaScript filters in the same route. Instead try to combine into a single JavaScript filter where possible and utilise the shared JS feature to reduce filter code complexity.
  - Do not use system commands, for example System.getProperty(). These calls are generally synchronised so that two calls cannot be made at one time.
  - Do not use System.exit() in any JavaScript filter as this will cause processing to halt.
  - Do not create delays in the code of a JavaScript filter. If a delay is needed use the message collector tab of the filter. Note that exact delays may not be achieved with the Message Collector. The defined collector components are polled periodically to find messages which meet the threshold for release, thus the actual delay is the combination of the timeout value and the polling interval
    - In particular the use of Thread.sleep() in a JS Filter should be avoided since this can lead to scenarios where all Route Executor Threads are stalled, and engine processing will pause
      - Use of Thread.sleep() or other delay techniques within the JS TCP Communication Points (where a solution requires this) is acceptable however, because the communication points have separate threads to the Routes and engine-wide processing is not impacted by these; only the thread sending / receiving data
  - Avoid excessive commenting – code developed following the above guidelines should be readable and self-documenting, comments should be used to identify major features, executions, or considerations of the code.

3.10.2 Use of Shared JS Library function

We recommend using shared JS Library functions wherever possible as this ensures a central location for management, testing, and updates (rather than requiring each JS filter that may contain a copy of the code to be updated). This in turn streamlines the code within shared JS filters and supports scenarios where multiple JS filters can be combined into a single filter without combining large amounts of JS code in the single filter.

When creating shared JS Library functions take the following into consideration:

- The shared JS Library function does not have scope visibility of the standard Objects available to the JS filter (such as the Input and Output message or the log) - ensure you pass these as parameters where required.
- Shared JS Library functions can only access other shared functions that are contained in the same Library in the current version.
    - While there are workarounds to this be aware they are not supported and not guaranteed to work in future versions
- For debugging during initial development, it is often better to build and test the function within a JS filter and then promote it to the JS Library (taking into consideration any scope dependencies as previously noted).
- From Rhapsody 6.8 it is possible to unit test shared JS Functions via assertions – this can be used to test self-contained functions that do not require access to the Message Objects – for example to test a function that formats a HL7 date to a SQL compatible date-time string as shown below:

Figure 26: **Example unit testing via assertions of a JS Function**

fn_convertHL7DatetimeToSQL Tests

```
/*function test_newTest() {
   fn_convertHL7DatetimeToSQL ("20220102103201");

} */
function test_emptyDate() {    // TODO:
    //assertUtil.assertIsTrue(true);
    //assertUtil.assertIsFalse(false);
    //assertUtil.assertIsEquals(1, 1);
    //assertUtil.assertIsNotEquals(1, 2);
    //assertUtil.assertIsNull(null);
    //assertUtil.assertIsNotNull(1);
    var result = fn_convertHL7DatetimeToSQL ("");
    assertUtil.assertIsEquals("", result);
}
function test_nullDate() {
    var result = fn_convertHL7DatetimeToSQL (null);
    assertUtil.assertIsEquals("", result);
}
function test_FullDate(){
    var result = fn_convertHL7DatetimeToSQL("20220119090249");
    assertUtil.assertIsEquals("2022-01-19T09:02:49", result);

}
function test_DateOnly(){
    var result = fn_convertHL7DatetimeToSQL("20220119");
    assertUtil.assertIsEquals("2022-01-19", result);

}
function test_shortDateTime(){
    var result = fn_convertHL7DatetimeToSQL("202201190902");
    assertUtil.assertIsEquals("2022-01-19T09:02:00", result);
```

Properties

3.10.3 Considerations for Multiple JS Filters regarding Parsing/Route Performance and storage

- The use of multiple JS filters can have an impact on the overall performance as each filter may require a parse of the message, thereby resulting in multiple parses (note this is not the case if there is no message definition on the route in question). Also each filter that changes the message body or properties requires a disk write to complete before the next step in processing continues – thus reducing the number of filters reduces impact to disk performance and storage use. In some cases, multiple JS filters may be required due to reasons such as:
    - Processing logic – for example after a message has been processed by a JS filter, it needs to be split because one path requires the message/properties at that state while the other path requires additional processing by a separate JS filter before output from the route.
    - Separating complex JS operations into discrete units of function to simplify testing and maintenance. However the following strategy can be used to achieve the same outcome:

- Move the unit into multiple shared JS Library functions (which are tested and validated separately and may be maintained separately) and then call the functions from within a single JS filter. This is the preferred method – particularly where the presence of a message definition on a route would otherwise cause multiple parses of the message.

3.11 Message Mapping Options and Considerations

3.11.1 General Considerations

Message mapping and transformation is supported through a number of Rhapsody features and components. In the past, the general rule was to use the Mapper as the compiled code was more efficient than some of the other options available. However, improvements in engine efficiency mean that while this is still generally true the difference in execution time is often insignificant and several other factors/constraints must now be considered when deciding how to manage message mapping as described below:

- The Mapper filter requires a mapper definition file, which in turn requires an input definition and output definition file for the message formats. The usage of the Mapper filter is generally better suited to scenarios where there is significant difference between the input and output format and where there is significant effort in mapping between them.
- However complex mappings may also be achieved in JS by accessing the parsed messages and use of the edi objects
- Note the Mapper does not support looping / multiple output message scenarios (e.g. Output 2 A2 for 1 input A13, for each OBX in an ORU output a new ORU message combining the common segments and each result). If this is required you will typically need to use a JS filter to split the output messages after mapping, so it is more efficient to simply do both in a single JS filter
- Mapping/Transformation that involves a small number of fields is better suited to the JavaScript filter (which may include use of the shared JS libraries) since the transformations can be grouped as execution lines and readily identified

when maintaining the solution. This is often the case when the input and output formats share a format definition.

- Even for cases where the definitions are different, the same outcome may be achieved by either:
  - for smaller transformations by using the HAPI libraries
  - Performing the logic changes and transformations using the input message definition then using string processing to restructure the message to the output system's requirement

- Mapping HL7 to XML or vice versa can be achieved through either the use of E4X template processing within the JS filter or by using the Mapper filter. Please consider the following exception scenarios:
  - When dealing with scenarios that require insertion of XML snippets into an existing XML document, these are currently best suited to delivery via mapping in JS filter using E4X processing.
  - When parsing large XML files while performing mapping to and from XML, these are currently best suited to delivery via mapping in Mapper filter.

Note e4x is only available in the JS v1 filter – i.e. up to and including v 6.10.1.

Later 7.x versions have the ability to handle XML processing, however the methods differ by version – please refer to the companion v7.x BPG documents for full details :

Version	XML Processing Support
7.0	• Xpath • Java DOM API See https://docs.rhapsody.health/rhapsody/rie/70/en/working-with-xml-messages.html
7.1	As Above plus the introduction of the XML Object API. This introduces an RIE native API that reproduces similar functionality to the e4x object – XPath expressions to read, traverse, build and modify XML messages in JavaScript. Global

functions are also available for XML escaping and tagged templating.

- Mapping where features required are only exposed through JS Objects will require the use of the JS filter – for example
  - HAPI
  - JSON[1]
  - lookupAll()
  - Snapshot loading and processing (especially where multiple snapshots are used)
- Similarly if you need to map using JSON Schema as a definition - this is currently only supported within the Mapper Filter.
- Mapping and transformation of messages that are known to not always conform to a strict message format are generally best handled within the JS filter (this is often true for like-for-like migrations of legacy interfaces where input messages may need to be processed without definitions to correct or retain format/value issues within the message) as there may be the potential to correct or ignore the error and continue processing.
  - Use of the Mapper in this scenario will either fail or ignore depending on the 'Fail on Errors" property configuration of the Mapper Filter.

3.11.2 Mapper Filter Specific considerations

- Use sub-maps for code reuse and easy maintenance.
- Structure your maps based on the output message structure.
- When using the Automap feature of the Map Designer care must be taken. This feature is best suited to message types where the input is very similar to the output; where this is not the case some components may not be auto mapped and "To Do" sections will exist which require manual coding for the mapping. Therefore, always ensure that if the Automap is used the code is searched globally for "To Do" sections and these are completed prior to use in

[1] As of version 6.8 Rhapsody has limited support for FHIR JSON within the Mapper, however there is no support for more general JSON

the engine.

3.12 Database Considerations

3.12.1 Where Possible Use the Database Communication Point in Preference to Filters

Communication points are much better at handling database connections due to their configurability relating to connection failures. They also have their own queues for processing which consequently do not block message flow and have a greater ability to ensure FIFO is maintained. In contrast, connection errors encountered within filters requires relatively more effort to ensure FIFO is not broken. Compared to a communication point which may simply retry the queued message.

- Use the Database communication point in Out->In mode instead of Database Lookup filter if possible.
- Use the Database communication point in input mode rather than Database Message extraction filter if possible.
- If a database filter is used, use Error Processing on the filter or via the Route Error Handler to handle error messages properly.
- Special consideration is required when using a database filter in multi-threaded routes Although FIFO guarantees that a message will leave a route in the same order as received it does allow multiple messages to be processed by filters concurrently. If the order of execution is important with respect to the state of data in the database, consider using FIFO-single threaded and/or setting the filter concurrency to 1. If it is possible to use a communication point for the same workflow setting the connections to 1 will ensure messages are processed in FIFO order

3.12.2 Choose the Correct Database Components

- If you want to retrieve data from a database table, use the Database communication point in input mode. It polls a database regularly to detect changes using a static query.
- If you want to insert or update one row into a database table for every

message it receives, use the Database Insertion communication point.

- If you want to retrieve data from a database table for every message it receives, use the Database Message Extraction filter, with understanding the caveats of the filter.
- If you want to insert or update data retrieved from a message and continue processing after that, use the Database communication point in Out->In mode or Database Lookup filter. They can also be used to insert directly into messages, populate message properties or replace an entire message body using retrieved values from a database table.

3.12.3 Use Configuration Properties to Handle Connection Failures Properly

- Set the query timeout limit by configuring the Query Timeout configuration property (only works for MS-SQL and Oracle) to prevent hanging database connections.
- Set the Socket Timeout configuration property to a non-zero value that matches a period longer than would be expected for a connection to succeed (the default value of zero means the socket connection will never timeout).
- Set the Idle Timeout configuration property for the Database communication point to drop inactive database connections automatically. We recommend using a timeout value less than an hour. However, this value is dependent on any active firewalls (e.g. if the firewall terminates inactive connections after 15 minutes). The value should be set to a lower value than the firewall threshold.
- If database filters are used, do not use an infinite value (0) for the Maximum Concurrency configuration property. Limiting the level of concurrency limits the number of concurrent database transactions and the memory used by the filter.
- When configuring FIFO enabled routes with database filters, bear in mind that a long running transaction will block the route and cause message build-up while it waits to commit, so where possible disable FIFO.

3.12.4 Use Notifications

Turn on the 'Message Potentially Stuck While Sending' notification for the Database communication point output mode and Out->In mode to get notified

when the sending message process does not complete within the specified time. This notification is turned on by default.

Turn on the 'Message Time in Filter Exceeded' notification for the Database Lookup filter and Database Message Extraction filter to get notified when message processing in the filter does not complete within the specified time. This notification is turned on by default.

3.12.5 Use Stored Procedures and Views in Preference to SQL Statements

Use of Stored Procedures and Views can simplify the configuration within the SQL editor of the component and the process of executing multiple statements.

Views should be used for lookup workflows; stored procedures should be used for update workflows – particularly where return values or multiple execution logic steps are required (such as execute an insert then retrieve the resultant row of data)

Calling a pre-configured view or stored procedure supports:

- Reuse of the query
- Simplified layout and parameter / return value passing
- Abstraction of database object names in the underlying query that may change per environment
- Ability for the DBA to assess the performance of the SQL statements and executing plans within their database environment

3.12.6 General SQL Scripting Standards

Below are the general conventions for SQL Scripting:

Table 3: **SQL Scripting Standards**

Item	Convention	Example

Metadata	• It is important that any SQL that is delivered to a customer is versioned and can be traced back to the originator.	`/*`
		`[Script Name]`
		`[Script Description]`
		
		`@author: Lyniate [Author Name]`
		`@created: yyyy-mm-dd`

	• The header to the right must be applied to any SQL that is created/delivered by Lyniate	`@reference: JIRA Number or External Reference (where relevant)`
		
		`!!!Important Notes!!!`
		
		`Version History:`
		`1.1 –`
		`1.0 – initial query`
		`*/`

Alias	• Always use aliases when the query involves more than one table.
	• Always specify the table aliases for columns in the SELECT clause.
	• Keep aliases short and unique – usually an abbreviation or acronym of the table is sufficient.
	• For commonly queried tables, keep a list of aliases for consistency.
		`SELECT [ep].[episodeId]`
		`FROM [dbo].[Episode] ep`
		`INNER JOIN [dbo].[Candidate] can ON [ep].[candidateId] = can.[candidateId]`

Identifiers	Always use identifier delimiters on column and table names. This ensures names are parsed correctly, particularly when they contain characters and reserved words that confuse execution. e.g. MS SQL uses [and] or ‘ and ‘	`SELECT [column1] AS 'col1'`
		`FROM [dbo].[Table1] t1`
		`INNER JOIN [dbo].[table2] t2`
		`ON t1.[columnName] = t2.[columnName]`
		`WHERE t2.[column3] like '%abc'`

Nested Selects	Avoid nested SELECT statements – these are computationally expensive as they execute for every result row	
Linked Server Queries	Avoid queries which use linked servers – these are inefficient as all of the results from rows and columns must	

	be delivered over the network before where clauses are applied locally	
Keywords	Always capitalise	Examples: SELECT, FROM, NOT NULL, CREATE, HAVING, ON, WHERE, JOIN, GROUP BY
Data Types	Always use lower case	int, varchar
Tabbing	• Ensure formatting is consistent and enhances readability. • Use a free formatter like Instant SQL Formatter or invest in SQL Prompt add-on to SQL Server	

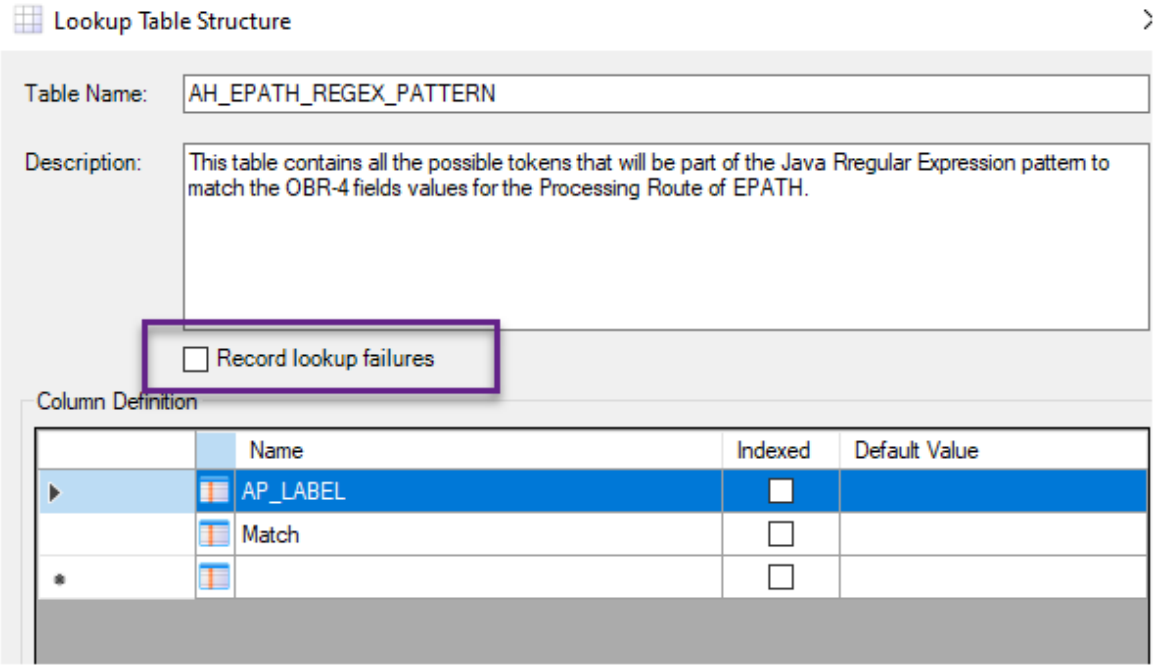
3.12.7 Rhapsody Lookup Tables

Lookup Tables within Rhapsody provide more flexible options and better performance than database filters such as the Generic Code Translation filter. They are typically suited to scenarios where there are not expected to be > 600,000 rows. In addition, they may be accessed directly from within JS and Mapper filters as part of message manipulation operations rather than requiring a separate component execution (communication point or filter).

Care must still be taken to ensure code which uses these features is safe, however as they are part of the engine runtime, they are less prone to errors as may be encountered with Database filters (such as connectivity failures).

If large numbers of lookup failures are expected, and are not expected to be routinely added to a table (for example a scenario where only a small number of input values are to be mapped, and if no lookup result is found the original value is retained) ensure that the option to record lookup failures is de-selected (it is selected by default):

Figure 27: **Record Lookup Failures**



3.12.7.1 lookupall() vs lookup() Return Values

The lookupAll() global JS method was introduced prior to v 6.5 (first version post-split from Orion). The current online documentation has been corrected, however versions of the product documentation prior to 6.5 and the related online heath may have an erroneous version of the feature where the example use uses the same return object-value as the lookup() function. This is not correct as the functions return different objects / values as follows:

3.12.7.1.1 lookup()

- Returns a Row object.
    - If there is one or more matches to the criteria the Object will hold the data from the first matching row
    - Values in the object are column names, the column values are accessed by specifying the column name as a member
    - The image below shows the use of the lookup() function, testing the result, falling back to a default if there is no result and then either throwing an error or using the value from the returned object's DOCCODE column

Figure 28: **Example lookup() call and return value use**

```
var luResult = lookup("MH_Nexus_To_SMR_DOC_Code", "OBR4", obr4, "SOURCE", nexusSource);
//if no result check for default value
if (!luResult){
    luResult = lookup("MH_Nexus_To_SMR_DOC_Code", "OBR4", "DEFAULT", "SOURCE", nexusSource);
}
if(!luResult){
    throw("Cannot find match for Document Code in 'MH_Nexus_To_SMR_DOC_Code' - check Default value is correct for " + nexusSource);
}

fileName = "" + mrn + "-SHN-" + luResult.DOCCODE + "-" + obr7
next.setProperty("FileName", fileName);
```

3.12.7.1.2 lookupAll()

- Returns an Array
    - If there are one or more matches, up to a default of 100[2] matches, each row is returned as an array member
    - If there are no matches the return value is an empty array, NOT a null object
        - Therefore you need to test against an array with length == 0 for no results rather than null
        - The error in the original (older-version) documentation was an example showing testing the result against null rather than an empty array

Figure 29: **Example use of lookupAll()**

```
  //set Interface Destination
 NoActiveDestination = "true";
  var n;
  var communicationPointName = next.getProperty("rhapsody:InputCommunicationPoint");

  var resultRows = lookupAll("MH_Interface_Broadcast_Mapping",
                    [{columnName: "source", value: communicationPointName},{columnName: "active", value: "true"}],
                     {returnColumns:["destinations", "message_trigger_event"]});


 if (resultRows.length >0) {
      for (var i = 0; i < resultRows.length; i++) {
        // Received the whole row from the table (all columns and values)
         var str = resultRows[i].message_trigger_event;
         n = str.indexOf(MsgType + "^" + triggerEvent);
         if(n != -1){
             next.addPropertyValue("router:Destination", resultRows[i].destinations);
             NoActiveDestination = "false";
         }
      }
  }
```

3.13 Developing Custom Components with the RDK

[2] This can be controlled via a rhapsody start-up property

Use of custom components should be as a last resort when the required function cannot be achieved using the standard Rhapsody communication points and filters. The default position is to always use the Rhapsody standard components if they can achieve the outcome due to the following reasons:

- Custom components may not provide the same level of detail of information to the logging and management console.
- They typically require a different skill set to produce and maintain when compared to standard Rhapsody components.
- They are not supported by Lyniate – any issues with custom components are the responsibility of the client to maintain and resolve.
- Reuse of existing customer libraries via the RDK can lead to issues due to
    - Java version issues (and compatibility of custom libraries that may have been used)
    - Execution of multiple functions which negates the message inspection, logging, and management aspects of Rhapsody

Reasons where a custom component may be required/considered where:

- Connectivity to a system or function is provided by a third-party software component (API, EJB etc.) that is not directly supported by current Rhapsody components – for example calling Oracle HMPI EJB functions to retrieve a patient Single Best Record.
- The functional requirement is of significant benefit to the solution or is a key requirement for the solution but is not able to be achieved by the current out-of-box components (for example development of signing and encryption filters using ADHA (Australian Digital Health Agency) libraries to upload documents to My Health Record).
- The functional requirement cannot be met with an existing Rhapsody component – for example connecting to shared drives of Echocardiology equipment dynamically from a trigger message to retrieve ECG Waveform PDF

When developing a custom component follow these processes:

- Raise a Jira ticket for development review of the proposed component development to ensure that:

- Similar functionality is not on the roadmap
- The proposal is sanity checked
- If the feature is beneficial to multiple customers or the product, there is an opportunity to uplift the component to become a standard feature of the product

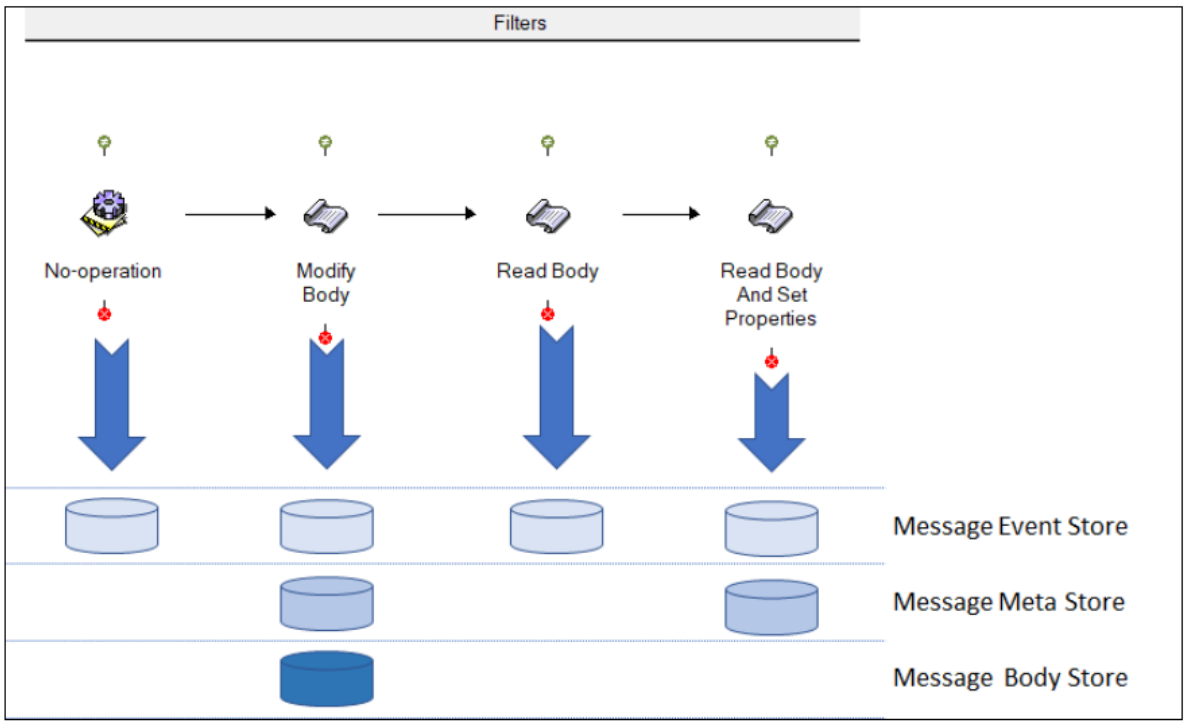
- Ensure the target functionality of the component is specific - the component should not generally encapsulate multiple unrelated functions (unless this is the requirement for the component such as may be required to manage a distributed transaction with orchestration). This is to streamline troubleshooting and maintenance efforts.
- Ensure the logging events are in line with standard Rhapsody components so that the information presented to the user within the Management Console is consistent with other standard components.

3.14 Message Properties

Message properties are applied to a message and are written to the meta store anytime they are added/deleted or changed. This can lead to scenarios where large amounts of disk space can be consumed (relative to the message bodies) because each time this store is updated all properties are written to disk, not just the updated properties.

The following diagram demonstrates how the sections of the data store are updated for this particular workflow:

Figure 30: **DataStore use for message flow through components**



Lyniate's best practice aims to reduce unnecessary writes to the data store as this has the dual benefit of both reducing IOPS requirements in high throughput engines and reducing the amount of disk storage required.

3.14.1 Avoid Assignment of Message Body to Property

In previous versions of Rhapsody (pre-6.2.1) there are some solution scenarios that would require capturing of the entire current contents of the message body as a property so that it could later be retrieved.

Message Properties are not intended to handle large amounts of data and doing so has a negative impact on performance as well as increasing the amount of data store used for each message operation. This can be avoided through one of the following methods:

- Use the Snapshot Save filter to save the current message and its properties. This can then be used to achieve the desired outcome by:
  - Using the Snapshot Load filter for simple scenarios where you want to replace the message on the path with the previously saved snapshot
  - Using the JS Object functions to load the snapshot or load a snapshot by ID

- While the Rhapsody:SnapshotMessageId property contains only the most recent saved snapshot, it is possible to save and retrieve multiple snapshots using custom properties and then load them through the JS loadMessageSnapshotForId method

- Save a temporary message into a database using a Database communication point (via stored procedure) and store the return value (a unique link to the stored message) as a message property. This can then be used by a downstream process to retrieve the saved message from the database and process it (also see the Database Considerations section within this guide). This is crucial because:
  - Message snapshots can only be loaded while they are present within the Rhapsody Data Store (within a single cluster only), i.e. while they have yet to be removed by the Archive Clean-up process or processed to a different cluster. In most scenarios, this is not an issue as a snapshot is not eligible for removal in the time frame where a typical sequence of route processes is executing in a single cluster.
  - However, there can be significant delay (days to weeks) between the process that stores a temporary message and the process that completes the output, or where the message is sent to another functional cluster via a Rhapsody Connector.

Note: a recent effort was undertaken for a large HIE customer experiencing performance issues. By replacing the process of storing message bodies temporarily as properties with the use of the Snapshot filter and JS filter the performance of these routes increased by 70%.

3.14.2 Managing Properties

As messages flow through the engine the properties are cumulative, this can result in "property spam" in later routes where large numbers of properties have to be managed/understood in downstream use. This can impact the effort required to analyse message property change within a message path in the Management Console. Large numbers of properties also impact the size of data store use and overall engine performance.

The following strategies should be employed to manage properties:

- If a value can be extracted from the message to achieve the required outcome (e.g. a logical comparison to only accept messages from a given source / facility) and this data does not change throughout the message

journey extract the field value at the time it is required within the filter logic, rather than setting as a property early and later retrieving it
    - This also has the advantage that Filter unit tests can be configured without the need to populate large numbers of property values
- Custom properties which are important for use in downstream routes deployed by a solution should be documented in a property register for easy reference by future teams maintaining or extending the solution (e.g., the set of properties set by the publisher of data which can be used by subscribers to determine if the data should be processed or filtered out).
- Custom properties received by a downstream route that are no longer required should be removed after use when possible (by setting the property value to null) so they do not continue to propagate
    - As this typically requires a filter this should be done when a given route stage would already be utilising a filter such as the JS filter which can update the properties rather than adding a filter solely for this purpose, unless the downstream properties are unmanageable in number.
- If many properties are required for downstream processing consider creation of a JSON object to contain all the "property": "value" pairs, this could be set as the message text and saved as a snapshot for use in downstream routes and filters rather than polluting the message properties with large numbers of properties.

Note a minimum set of properties are created by the engine for various message operations such as the input time the message was received, input and output message types for parsed messages (as well as the definition name) and so on. Different components will set different standard properties. The full list of published properties, and where they are set, is found at https://docs.lyniate.com/rhapsody/rie/69/en/message-properties.html#published-message-properties

3.14.3 Indexing and Searching

By default, most Rhapsody properties are not directly searchable within the Management Console. Message properties can be defined as indexed to be used

for message searches in Rhapsody Management Console based on the content of this property, obviating the need to a full meta search across all messages.

In order to prevent unnecessary processing and disk utilisation by the message property indexing logic, ensure indexing is restricted solely to fields that are required for fast lookups on message searches.

Note the following indexed properties are set by the engine the first time a message is parsed by a definition on receipt by a route:

- DefinitionName
- rhapsody:InputMessageType
- rhapsody:OutputMessageType

These may also be set and indexed where a Message Tracking scheme is applied to an Out->In Communication Point:

- rhapsody:MessageControlId (shown in MC as Message Control Id) – note this is the combination of fields selected for matching in the tracking scheme so may not necessarily equate to MSH-10
- Tracking State

3.15 Rhapsody Variables

Use Rhapsody Variables for environment specific parameters such as:

- File paths/directories
- Email addresses
- Remote system host and port details
- Local service host and port details
- Connection details for external systems and databases such as username
- Web service URLs
- Access credentials

Rhapsody variables allow for differences in system configurations between environments and provide a smooth transition of configuration to the production environment as the variables are not overridden by default when Rhapsody configuration is imported from one environment to another. This allows different

configuration settings to be used in production and non-production environments. The use of variables also helps expose configurations settings that would otherwise be embedded and hidden, making for quick access and change.

3.16 Testing and Peer Review

Peer review is recommended during development and is required before promotion of a solution between environments or merging into a master repository.

Peer review should take into consideration the documentation (including route self-documentation, test results and any documented deviations or design decisions which are not normally considered best practice). The reviewer is typically not the same resource who develops the solution.

3.16.1Development Testing

A solution will require testing and validation to demonstrate the soundness of the solution. Evidence of the completion and result of testing will typically be required by the peer review process and ultimately the customer. The level of testing and related documentation may differ by customer/contract however evidence of completion of the following is expected at minimum before promotion out of the development environment:

- Unit testing of filters and conditional connectors via the inbuilt test feature for:
  - Invalid cases
  - Valid cases
  - Both of which may require permutations based on the number of conditions or branches executed in a given component
- Unit testing of message flow (input, output, property and message change throughout the route as per the message flow in the Management Console) for individual routes
  - For both valid and invalid messages, messages which demonstrate

conditions of the route end to end.

- End to end message testing for the sequence of routes related to a given interface or set of interfaces for the current solution under development.

Note the following:

- In some cases, it is not effective to test certain conditions at a unit level within the filter test function – particularly if the filter requires the result of processing from earlier route components; in these cases, the testing will revert to the route level unit testing.
- Where a filter test requires a message snapshot the configured property must contain the ID of a message still within the message store if tested within the filter. This may require the use of a hold queue to ensure the identified message is available.
- Database filters can be used to unit test, including validation of parameter population, the required configuration prior to applying the same configuration to a communication point (as communication points do not currently have the capability to perform this type of test
- Greenfield interfaces frequently will not have "expected output" messages to test against. In these cases, unit testing may initially utilise a reverse-defect methodology whereby the input message is set as the expected output, and the test then verifies the filter test flags the expected update field as a defect, and that the field contains the expected value

3.16.2 End to End / Route Testing

End to end testing ensures the interface produces the required outcome from receipt of message to delivery to downstream systems.

- Migration Projects will typically follow a process of replaying input logs of the legacy engine against the equivalent Rhapsody interfaces, then using stub systems to receive and log the output from Rhapsody which is then compared to the equivalent output from the legacy engine

- The volume of messages required for these types of projects makes them unsuitable for use with the Route testing API due to the effort in configuring the individual test cases
- In some cases, inspection of the result of database calls via tools such as SSMS and BeyondCompare may be required to verify the result of database calls. More involved testing may require involvement of customer DBAs and is generally beyond the scope of testing within Lyniate's project responsibilities

- Greenfield / New interface projects typically do not have a large volume of messages to test with, frequently a set of input messages may be available and no output messages. In this scenario sample/dummy messages with data meeting workflow requirements will be used to verify:
  - End to end delivery of messages / filtering out of messages is correct per interface by tracking the message through the MC / capturing output and validating expected number and type of message

Rhapsody 6.6 introduced the Route testing framework via the Rest API (https://docs.lyniate.com/rhapsody/rie/66/en/testing-routes.html) however the effort required to configure the individual test cases is far greater than the existing manual methods.

Combined with the frequent lack of test messages, and the need to manually create test cases, from a pragmatic perspective the 6.6 testing framework is best suited to CI/CD workflows where the same set of tests will be executed for any updates / changes to ensure the existing functionality has not been compromised by an update to configuration or software versions.

3.17 Other/General Considerations

3.17.1 Assign a Specific User to Run the Rhapsody Process

On the server where Rhapsody is installed, instead of running the application service as the default administrator user, set up a separate user to run the Rhapsody service. There are two key benefits to this approach:

- Enable detection of issues that relate to operating system and Rhapsody interactions and for configuration of operating system and environment

settings specific to Rhapsody. For example, setting the maximum number of file handles able to be assigned to the Rhapsody process.

- Ensure that resources assigned to the Rhapsody execution user are not shared with other processes that are not related to the Rhapsody Engine on the server system.

Note: The service account should have access to the requisite network resources and the password policy for the account should be set to never expire.

3.17.2 Use Individual Accounts for Access to the IDE and Management Console

Avoid re-using the default administrator account for multiple users as this will make auditing and tracking of user actions difficult. Ensure that a new account is created for access (either via LDAP or the inbuilt Rhapsody account management), with appropriate roles assigned for each new user. This will enable:

- Restriction of access to IDE features or Management Console functions by role.
- Ability to easily disable a user's access when required.
- Ability to determine the user responsible for an action based on their user ID (e.g., who checked in a given change to the IDE).

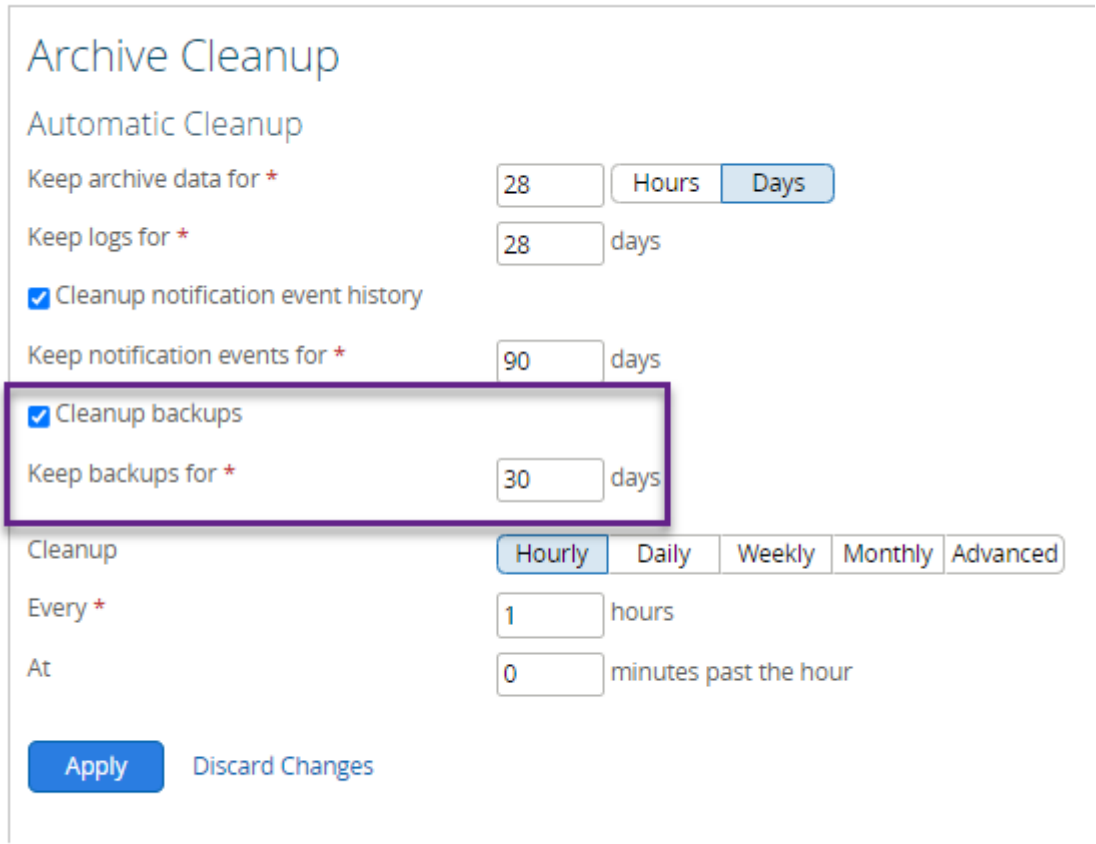
3.17.3 Backups

- Consider when to schedule backups and what areas are required for backups.
    - Backup of the Message Store will pause routes for a period of time to backup active files, in high volume throughput sites this can cause noticeable delay to the sending clients.
    - Message Store data is transient and only retained in the datastore after delivery for BAU purposes, as such there is typically little benefit in backing up Message Data which may be stale by hours or days in the event of a restore-from-backup. Typically, Lyniate does not recommend backing up the message store. Other methods such as block level replication may be used for DR or backup and recovery
- Configure an infrequent full back up to the same directory as a frequent incremental; for example, a weekly full backup and a daily incremental backup.
- Each full backup provides a fresh starting point for further incremental

backups, rather than having a never-ending chain.

Note: Backups in versions of Rhapsody prior to 6.5 were not removed by the engine and required customer management to ensure exhaustion of the target disk did not occur. From Rhapsody 6.5, an option is available within the Archive Cleanup section of the management console which customers can select to deleted backups older than a certain period. Note this item is NOT selected by default, and should be enabled with a period meeting the customers requirement:

Figure 31: **Backup Cleanup Option**



3.17.4 Archive Cleanups

- Schedule routine operations of the Archive Cleanup.
- Schedule Error Queue Defragmentation process.

3.17.5 Configure Management Console Notification Schemes

The configuration of the notification schemes varies highly from customer to customer depending on their size, priorities of different interfaces and the number of resources in the team administering Rhapsody. At a bare minimum, the following should be configured and tested per environment:

- Ensure configuration is completed to send notifications to a default administrative user email account or distribution list name.
- Ensure that configuration of thresholds for warnings and alerts (e.g. large queues, communication point shut down etc.) has been configured in line with the minimum solution/customer requirements.
- Ensure that notification event history is included in the scheduled Archive Cleanup configuration as this can build up over time and take up considerable disk space

4 Virtualisation Best Practice

Please refer to the companion document - Rhapsody Virtualisation Best Practices guide.